

SMART GLOVE FOR INTELLIGENT DEVICE CONTROL

A

Project Report

Submitted in partial fulfillment of the requirement for the award of

Degree of

Bachelor of Computer Applications

Submitted to

310 – CIMAGE PROFESSIONAL COLLEGE



Submitted By

ADARSH SARTHAK

Registration No.23303310121

BCA-6th Sem

Session-2023-2026

Under Joint Guidance of

Mr. Niraj Kumar Singh &

Er. Pankaj Chandra Sinha

[Asst. Professor]

Computer Science &

Information Technology



CIMAGE PROFESSIONAL COLLEGE
Department of Computer Science & Information Technology
Session 2023-2026

DATE:.....

CERTIFICATE

This is to certify that the work embodies in this project entitled, **“SMART GLOVE FOR INTELLIGENT DEVICE CONTROL”**, being submitted by **ADARSH SARTHAK** (23303310121) in partial fulfillment of the requirement for the award of **“Bachelor of Computer Applications”** to **ARYABHATTA KNOWLEDGE UNIVERSITY, PATNA** during the academic year 2022-25 is a record of bonafide piece of work, carried out by him/her under our/my supervision and guidance in the **“Department of Computer Science & information Technology”**, **CIMAGE PROFESSIONAL COLLEGE**.



Dr. Neeraj Agrawal
Director

Dr. Amit Kumar Shukla
Head of Department-IT



CIMAGE PROFESSIONAL COLLEGE

Department of Computer Science & Information Technology

Session 2023-2026

Date:.....

APPROVAL CERTIFICATE

The project report entitled **“SMART GLOVE FOR INTELLIGENT DEVICE CONTROL”**, being submitted by

ADARSH SARTHAK (23303310121) has been examined by us and is hereby approved for the award of degree **“Bachelor of Computer Applications”**, for which it has been submitted. It is understood that by this approval the undersigned do not necessarily endorse or approve any statement made, the opinion expressed or conclusion drawn therein, but approve the project only for the purpose for which it has been submitted.



(Internal Examiner)

(External Examiner)

Date:

Date:



CIMAGE PROFESSIONAL COLLEGE

Department of Computer Science & Information Technology

Session 2023-2026

Date:.....

DECLARATION

I, **ADARSH SARTHAK** hereby declare that the work, which is being presented in the project report entitled **“SMART GLOVE FOR INTELLIGENT DEVICE CONTROL”**, partial fulfillment of the requirements for the award of degree of **“Bachelor of Computer Applications”** submitted in the department of **Computer Science & Information Technology (CIMAGE PROFESSIONAL COLLEGE)** is an authentic record of my own work carried under the joint guidance of **Niraj Kumar Singh Sir & Pankaj Chandra Sinha Sir**. I have not submitted the matter embodied in this report for the award of any other degree.

ADARSH SARTHAK

Registration No : **23303310121**

BCA 6th Sem

Session-2023-2026

ACKNOWLEDGEMENT

“A journey is easier when you travel together. Interdependence is certainly more valuable than independence.”

I would like to thank, **Niraj Kumar Singh Sir & Er. Pankaj Chandra Sinha Sir**, for providing regular guidance and insight into my project work. I also thank them for all advice he has given me in the past year, and for always having time for me, whenever I needed.

I give special thanks to **Dr. Amit Kumar Shukla Sir, Raju Upadhyay Sir, and Ravi Kumar Soni Sir, Anjesh Sir** for always being willing to help find solutions to any problems I had with my work.

“The completion of any project depends upon the cooperation, coordination, and combined efforts of several resources of knowledge, inspiration, and energy”.

I also extend my deepest gratitude to **Director, CIMAGE GROUP OF INSTITUTIONS, PATNA, Dr. Neeraj Agrawal Sir, Dean CIMAGE GROUP OF INSTITUTIONS, PATNA, Dr. Neeraj Kumar Sir and Centre Head Mrs. Megha Agrawal Ma'am** for providing all the necessary facilities and true encouraging environment to bring out the best in my endeavors.

I express my gratitude and thanks to all the staff members of Computer Science departments for their sincere cooperation in furnishing relevant information to complete this Project well in time successfully.

I extend a special word to my friends, who have been a constant source of inspiration throughout my project work.

Lastly but not least I must express my cordial thank to my parent and family members who gave me the moral support without that it is impossible to complete my project work. With this note, I thank everyone for the support.

(ADARSH SARTHAK)

Registration No : <23303310121>

BCA 6th Sem

Session-2023-2026

CONTENTS

REPORT.....	
-------------	--

Contents

SMART GLOVE FOR INTELLIGENT DEVICE CONTROL	1
310 – CIMAGE PROFESSIONAL COLLEGE	1
CIMAGE PROFESSIONAL COLLEGE	1
Department of Computer Science & Information TechnologySession 2023-2026	1
CIMAGE PROFESSIONAL COLLEGE	2
CIMAGE PROFESSIONAL COLLEGE	3
Session 2023-2026.....	3
CONTENTS	5
INTRODUCTION	9
OBJECTIVE OF THE PROJECT	10
PROJECT OVERVIEW	10
PROBLEM DEFINATION	12
NEED FOR PROJECT	13
LITERATRE REVIEW	14
EXISTING SYSTEM	15
PROPOSED SYSTEM	16
ADVANTAGES OF THE PROPOSED SYSTEM	18
PROJECT PLANNING AND SCHEDULLING	18
GANTT CHART	19
PERT CHART	20
FEASIBILITY STUDY	20
TECHNICAL FEASIBILITY	20

ECONOMIC FEASIBILITY.....	21
OPERATIONAL FEASIBILITY	21
REQUIREMENT SPECIFICATION	21
HARDWARE REQUIREMENTS.....	21
SOFTWARE REQUIREMENTS	22
Programming Languages Used	23
TOOLS, TECHNOLOGIES AND PLATFORMS USED	23
SYSTEM ARCHITECTURE	31
SYSTEM DESIGN AND WORKING METHODOLOGY	35
HARDWARE IMPLEMENTATION	43
COMMUNICATION	50
Glove → ESP32.....	51
ESP32 → ESP32	52
Receiver → Host PC	53
Host PC → Docker	54
Docker → Syncro5 Cobot	55
HARDWARE DESCRIPTION	56
SOFTWARE DESCRIPTION	66
PROGRAM DEVELOPMENT	71
ESP32 Transmitter	71
Python Communication	78
C++ Robot Controller.....	81
Gesture Recognition and Command Mapping.....	82
Data Flow Diagram (DFD)	90
Flowcharts	96
Module Description	107
Implementation	115

Testing and Validation.....	121
Problems Faced and Solutions.....	129
Performance Analysis.....	134
Applications.....	137
Limitations.....	139
Future Scope and Enhancements.....	140
References and Appendix.....	141
References.....	141
Circuit Diagram.....	143
Connection Diagram.....	143
Source Code.....	144
Snapshots.....	145
Conclusion.....	145

SMART GLOVE FOR INTELLIGENT DEVICE CONTROL

INTRODUCTION

The advancement of automation and robotics has increased the demand for smart and user-friendly control systems. Traditional methods of controlling robots and electronic devices often require specialized controllers or programming knowledge, making them less convenient for general users.

The **Smart Glove for Intelligent Device Control** is a wearable system that enables users to control intelligent devices using simple hand gestures. The glove uses flex sensors to detect finger movements, while an ESP32 microcontroller processes the sensor data and transmits commands wirelessly through the ESP-NOW protocol.

In this project, the received commands are processed by a Host Computer and transferred to the Syncro5 Collaborative Robot through an Industrial PC running inside a Docker environment. A C++ application interprets these commands and controls the robot's movement and gripper in real time.

Although the system is developed primarily for controlling the Syncro5 Cobot, it can also be adapted to control other intelligent devices such as RC cars, robotic arms, mobile robots, and IoT-based systems with minor software modifications.

The main objective of this project is to develop a low-cost, reliable, and intuitive gesture-based control system that enhances human-machine interaction and provides a flexible platform for future intelligent automation applications

OBJECTIVE OF THE PROJECT

The main objective of this project is to design and develop a Smart Glove for Intelligent Device Control that enables users to operate electronic and robotic devices through simple hand gestures. The specific objectives of the project are:

To develop a wearable smart glove using flex sensors for gesture detection.

To recognize hand gestures and convert them into control commands.

To establish reliable wireless communication using ESP32 and ESP-NOW technology.

To control the Syncro5 Collaborative Robot through hand gestures.

To perform real-time movement and gripper operations based on user commands.

To provide a simple and intuitive human-machine interaction system.

To reduce the dependency on traditional controllers and manual programming.

To develop a low-cost and scalable control solution for intelligent devices.

To create a system that can be extended for controlling RC cars, robotic arms, drones, and other smart devices.

To provide a foundation for future research in gesture-based automation and intelligent control systems.

PROJECT OVERVIEW

The Smart Glove for Intelligent Device Control is a wearable gesture-based control system developed to operate intelligent devices through natural hand movements. The system uses five flex sensors mounted on a glove to detect finger bending and generate predefined control commands. These commands are processed by an ESP32 microcontroller and transmitted wirelessly to another ESP32 receiver using the ESP-NOW communication

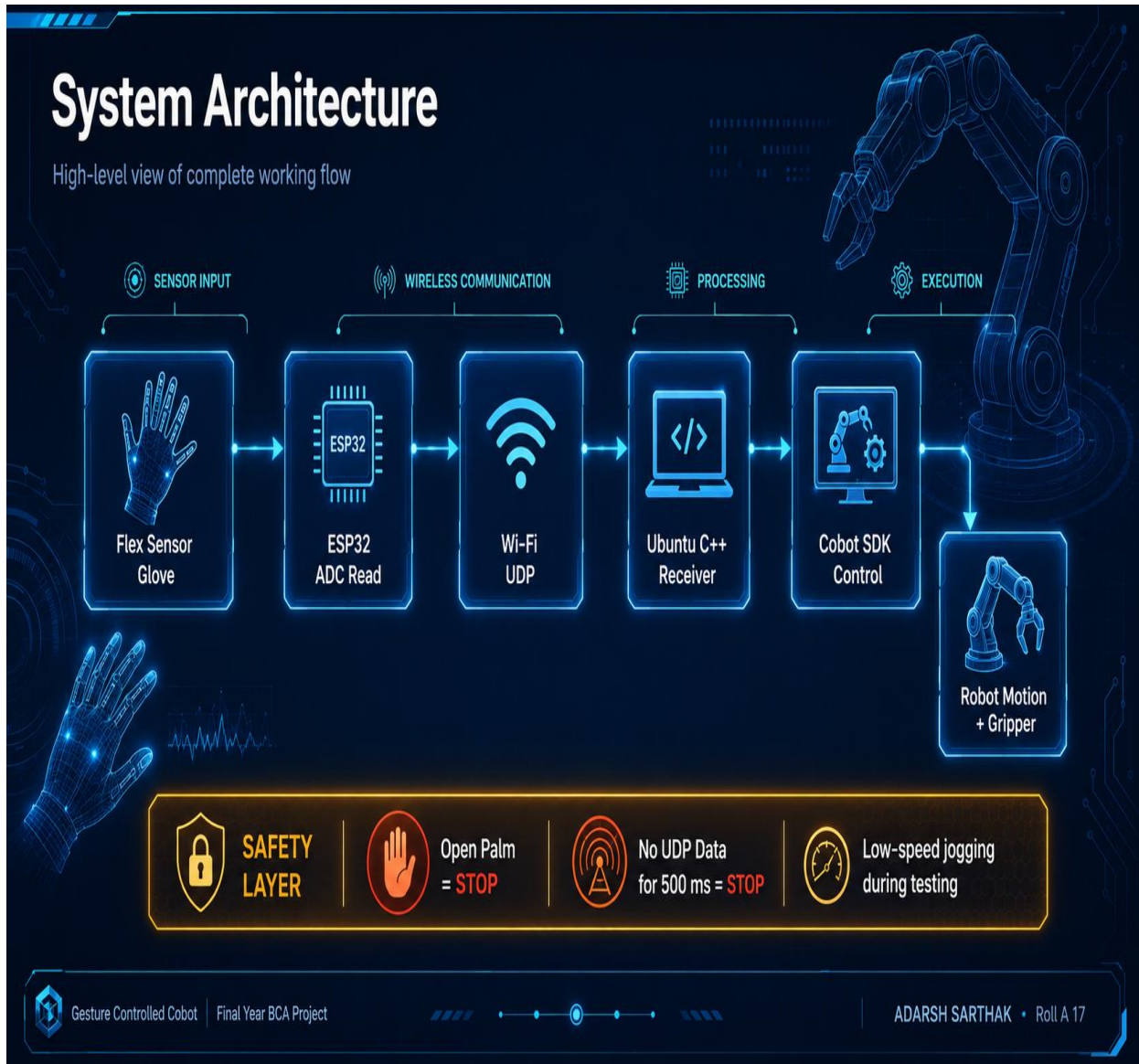
protocol.

The receiver sends the received commands to a Host Computer through USB serial communication. A Python application running on the Host Computer processes the commands and transfers them to the Industrial PC using SSH. Inside the Docker environment, a C++ control program continuously reads these commands and controls the movement of the Syncro5 Collaborative Robot and its gripper in real time.

The developed system provides a reliable, low-latency, and user-friendly method for controlling robotic systems without using conventional controllers or complex programming interfaces.

Although the current implementation is demonstrated using the Syncro5 Cobot, the same smart glove can be adapted to control other intelligent devices such as RC cars, robotic arms, drones, wheelchairs, and IoT-based systems with minimal software modifications.

The project integrates embedded systems, wireless communication, industrial robotics, and software development to create an efficient human-machine interaction platform suitable for industrial, educational, and research applications.



PROBLEM DEFINATION

Collaborative robots and other intelligent devices are commonly controlled using teach pendants, joysticks, keyboards, or specialized software. These methods often require technical knowledge, physical interaction with the controller, and additional training, making them less convenient for users in dynamic working

environments.

There is a need for a simple, portable, and intuitive control system that allows users to operate intelligent devices naturally through hand gestures. Such a system should provide reliable wireless communication, real-time response, and ease of operation without relying on complex control interfaces.

The Smart Glove for Intelligent Device Control addresses this problem by using flex sensor-based gesture recognition to generate control commands. The system enables users to control the Syncro5 Collaborative Robot and other intelligent devices through simple hand movements, providing an efficient, user-friendly, and cost-effective solution for human-machine interaction.

NEED FOR PROJECT

The increasing use of intelligent devices and collaborative robots has created the need for simpler and more intuitive control methods. Conventional control systems often depend on specialized hardware or programming skills, which can reduce flexibility and increase the learning time for new users.

The Smart Glove for Intelligent Device Control was developed to provide a natural and efficient way of controlling devices through hand gestures. By using flex sensors and wireless communication, the system allows users to operate intelligent devices without physical controllers or complex programming.

The project improves human-machine interaction by providing real-time gesture recognition, wireless operation, and ease of use.

Although the system is demonstrated using the Syncro5 Collaborative Robot, it can also be adapted for controlling RC cars, robotic arms, drones, wheelchairs, and other intelligent devices.

This project also serves as a foundation for future developments in wearable technology, smart automation, and gesture-based control systems.

LITERATURE REVIEW

Gesture-based control systems have gained significant attention in recent years due to their ability to provide natural and efficient human-machine interaction. Researchers have developed various techniques using cameras, wearable sensors, and computer vision to control robots and intelligent devices.

Flex sensor-based smart gloves have become a popular solution because they accurately detect finger movements while being lightweight, portable, and easy to use. Compared to vision-based systems, wearable gloves are less affected by lighting conditions and provide more consistent gesture recognition.

The ESP32 microcontroller has been widely adopted in embedded applications because of its built-in Wi-Fi capability, low power consumption, and support for the ESP-NOW communication protocol, which enables fast and reliable wireless communication between devices.

Collaborative robots such as the Syncro5 Cobot are designed to work safely alongside humans and are increasingly used in industrial automation, education, and research. Integrating wearable smart gloves with collaborative robots provides a more intuitive and flexible method of robot control, reducing the need for conventional controllers and complex programming.

The present project combines flex sensors, ESP32, Python, Docker, SSH communication, and the Syncro5 Robot SDK to develop a reliable gesture-based control system for intelligent devices.

EXISTING SYSTEM

At present, most collaborative robots and intelligent devices are controlled using teach pendants, joysticks, keyboards, or dedicated software interfaces. These methods require physical interaction with the control device and often demand technical knowledge for operation and programming.

Vision-based gesture control systems are another existing approach, where cameras are used to recognize hand gestures. Although effective, these systems are sensitive to lighting conditions, camera positioning, background noise, and occlusions, which can affect their accuracy and reliability.

Some wearable control systems are also available, but many of them are expensive, application-specific, or require complex hardware and software integration.

Due to these limitations, existing systems may not provide a simple, portable, and cost-effective solution for controlling intelligent devices in real time.

The proposed Smart Glove for Intelligent Device Control overcomes these limitations by using flex sensors, ESP32-based wireless communication, and a lightweight software architecture to provide an efficient and user-friendly gesture-based control system.

Limitations of Existing System :-

- Requires specialized controllers or teach pendants.
- Needs technical knowledge for operation.
- Camera-based systems are affected by lighting and background conditions.
- Limited portability and flexibility.
- Higher implementation cost in some solutions.
- Less suitable for real-time wearable interaction.

PROPOSED SYSTEM

The proposed Smart Glove for Intelligent Device Control is a wearable gesture-based system designed to control intelligent devices through natural hand movements. The system uses five flex sensors mounted on a glove to detect finger bending and convert these gestures into predefined control commands.

An ESP32 microcontroller processes the sensor data and transmits the commands wirelessly to another ESP32 receiver using the ESP-NOW communication protocol. The receiver forwards the commands to the Host Computer through USB serial communication. A Python application reads the incoming commands and securely transfers them to the Industrial PC using SSH. Inside the Docker environment, a C++ application continuously monitors the command file and controls the Syncro5 Collaborative Robot in real time.

The proposed system provides a simple, reliable, and low-cost solution for gesture-based control. It eliminates the need for conventional controllers while improving the ease of operation and flexibility. Although implemented for the Syncro5 Cobot, the same system can be adapted to control RC cars, robotic arms, drones, wheelchairs, and other intelligent devices by modifying the control software.

Features of the Proposed System :-

- Wearable smart glove with flex sensors.
- Real-time gesture recognition.
- Wireless communication using ESP-NOW.
- Secure data transfer through SSH.
- Real-time control of the Syncro5 Cobot.
- Low-cost and user-friendly design.
- Scalable for controlling multiple intelligent devices.

PROCESS LOGIC

From sensor reading to cobot movement



ADVANTAGES OF THE PROPOSED SYSTEM

The Smart Glove for Intelligent Device Control offers several advantages over traditional control methods by providing a simple, wireless, and intuitive way of operating intelligent devices.

Advantages

Provides natural and easy control through hand gestures.

Eliminates the need for keyboards, joysticks, or teach pendants.

Enables real-time wireless communication using ESP-NOW technology.

Low-cost and portable compared to many existing control systems.

Improves human-machine interaction through wearable technology.

Easy to operate with minimal training.

Supports real-time control of the Syncro5 Collaborative Robot and its gripper.

Can be extended to control RC cars, robotic arms, drones, wheelchairs, and other intelligent devices.

Modular design allows easy hardware and software upgrades.

Suitable for educational, research, industrial, and automation applications.

The proposed system combines embedded hardware, wireless communication, and software technologies to provide an efficient, reliable, and flexible solution for intelligent device control.

PROJECT PLANNING AND SCHEDULLING

Proper planning and scheduling were carried out to ensure the successful completion of the project within the given academic timeline. The project was divided into different phases, including research, hardware development, software development, communication setup, testing, debugging, and final documentation. Each phase was completed systematically to achieve the desired objectives.

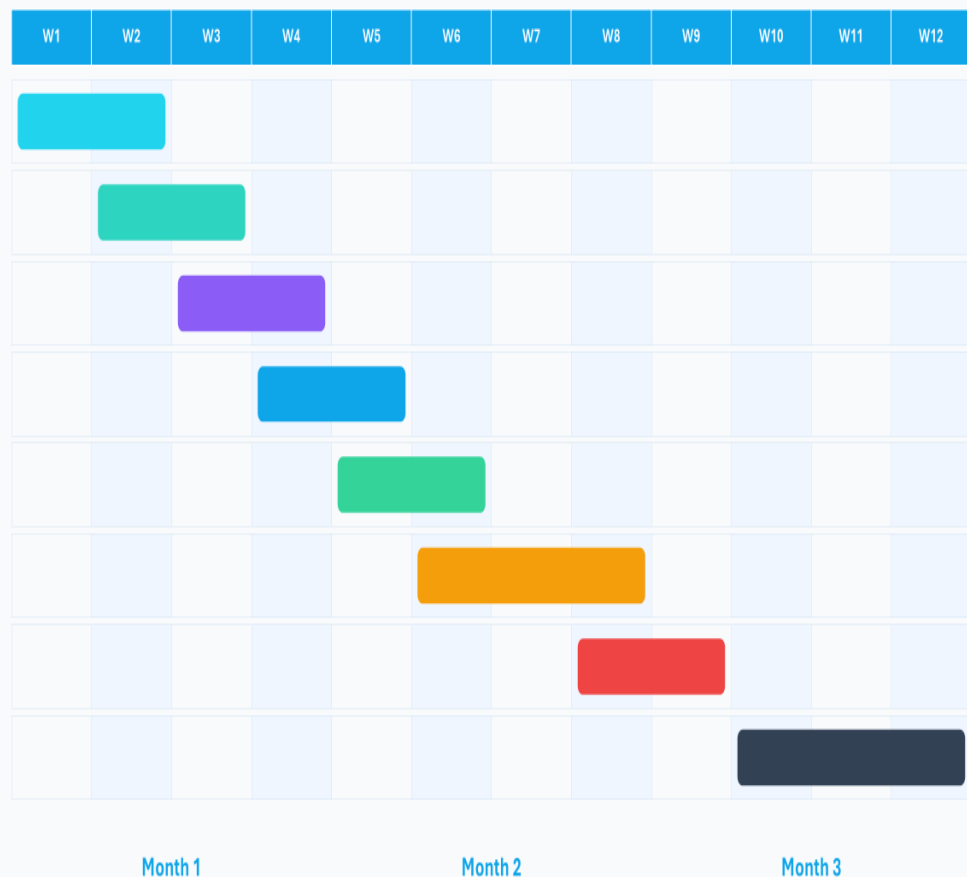
The development of the project was carried out from May 2026 to July 2026. During this period, both hardware and software components were integrated and tested to ensure reliable communication and smooth operation of the system.

The overall project schedule is represented using the Gantt Chart and PERT Chart, which illustrate the sequence of activities, their duration, and their dependencies throughout the project lifecycle.

GANTT CHART

Gantt Chart: 3-Month Project Schedule

Planned execution timeline



PERT CHART

10.2 PERT Chart

Suggested flow:

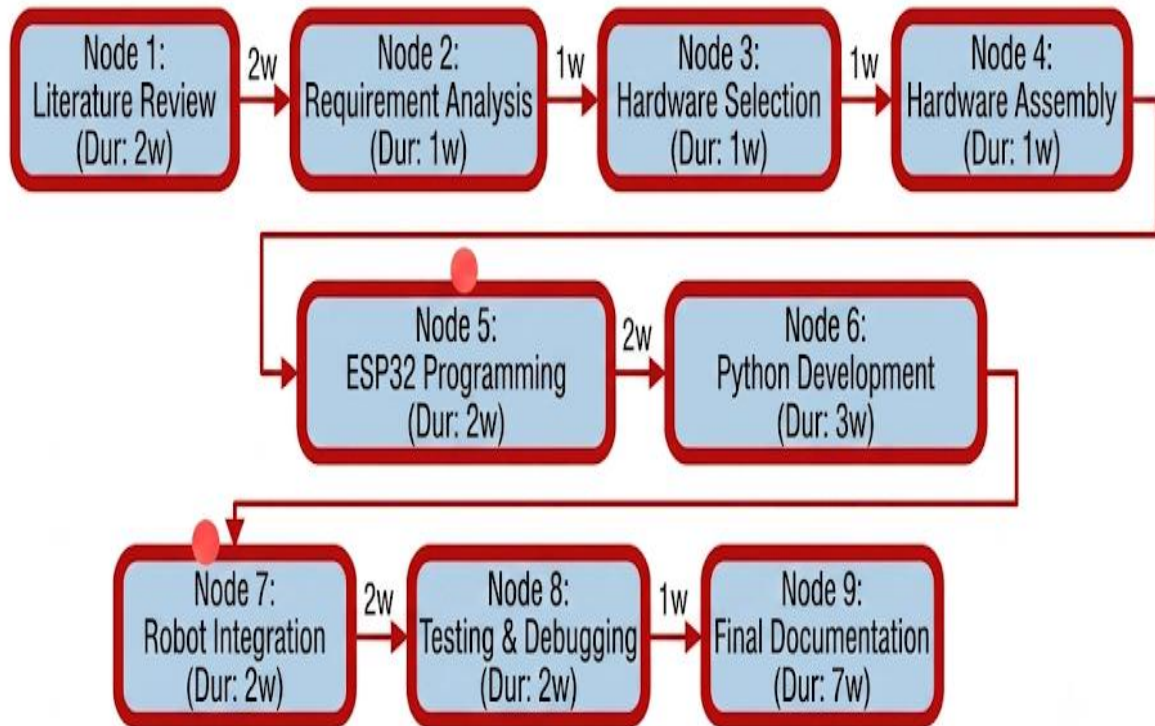


Figure 10.2: PERT Chart of Project Development.

FEASIBILITY STUDY

A feasibility study was conducted to evaluate whether the proposed system could be successfully developed and implemented using the available resources, technology, and budget. The study confirmed that the project is practical, cost-effective, and suitable for real-world applications.

TECHNICAL FEASIBILITY

The project is technically feasible because it is developed using readily available hardware and software components such as ESP32 microcontrollers, flex sensors, Python, C++, Ubuntu, Docker, and the Syncro5 Robot SDK. The integration of these technologies enables reliable gesture recognition, wireless communication, and real-time robot control.

ECONOMIC FEASIBILITY

The proposed system is economical as it uses low-cost electronic components and open-source software tools. Most of the software, including Python, Arduino IDE, Ubuntu, Docker, and GCC, is freely available, reducing the overall development cost. The project can be implemented without expensive control equipment.

OPERATIONAL FEASIBILITY

The system is simple to operate and requires minimal user training. Hand gestures provide a natural and intuitive way of controlling intelligent devices, making the system suitable for educational, research, and industrial applications. The wireless communication and real-time response improve the overall usability of the system.

The feasibility study confirms that the Smart Glove for Intelligent Device Control is technically reliable, economically affordable, and operationally efficient, making it a practical solution for gesture-based intelligent device control.

REQUIREMENT SPECIFICATION

The successful implementation of the Smart Glove for Intelligent Device Control requires appropriate hardware and software components. The selected components ensure reliable gesture detection, wireless communication, and real-time control of intelligent devices.

HARDWARE REQUIREMENTS

Component	Specification	Purpose
Syncro5 Collaborative Robot	6-DOF Cobot	Robot Control
Feetech Gripper	Electric Gripper	Object Grasping
ESP32 (2 Units)	Wi-Fi Microcontroller	Transmitter & Receiver
Flex Sensors (5 Units)	2.2-inch	Finger Gesture Detection
Leather Glove	Wearable Glove	Sensor Mounting
Lithium-ion Batteries	3.7V Rechargeable (2 Units)	Power Supply
Buck Converter	DC-DC Converter	Voltage Regulation
3S Li-ion Charging Module	Type-C Charging Module	Battery Charging
10 kΩ Resistors	Fixed Resistors	Flex Sensor Circuit
USB Cable	USB Data Cable	Receiver Communication
Industrial PC (Odyssey)	Ubuntu-Based System	Robot Controller
Host PC	Ubuntu 22.04	Gesture Processing
Jumper Wires	Connecting Wires	Circuit Connections
Wi-Fi Router (Optional)	Wireless Network	SSH Communication

SOFTWARE REQUIREMENTS

Software	Version / Platform	Purpose
Ubuntu Linux	22.04 LTS	Operating System
Docker	Latest Stable	Robot SDK Environment
Python	3.10	Communication Program
C++	C++17	Robot Control Program
Arduino IDE	Latest Version	ESP32 Programming
CMake	Latest Version	Project Compilation

GCC / G++	GNU Compiler	C++ Compilation
SSH	OpenSSH	Secure Communication
ESP-NOW	ESP32 Protocol	Wireless Data Transfer
Syncro5 Robot SDK	Addverb SDK	Robot Control
Git	Version Control	Source Code Management

Programming Languages Used

The project was developed using multiple programming languages, each serving a specific purpose:

C++ – Robot motion control and gripper operation.

Python – Serial communication, command processing, and SSH-based file transfer.

Arduino (Embedded C/C++) – Programming the ESP32 transmitter and receiver.

Shell Commands (Linux) – Docker management, SSH connection, compilation, and execution.

The selected hardware and software provide a stable, efficient, and scalable platform for developing the Smart Glove for Intelligent Device Control. The modular architecture also allows future upgrades without major modifications to the existing system.

TOOLS, TECHNOLOGIES AND PLATFORMS USED

The successful development of the Smart Glove for Intelligent Device Control required the integration of embedded hardware, software development tools, communication protocols, and robotic control platforms. Each technology was selected based on its performance, reliability, compatibility, and suitability for real-time gesture-based device control.

The project combines wearable sensing technology, wireless communication, embedded programming, industrial robotics, and software development into a

single integrated system. This chapter describes the purpose and contribution of each tool and technology used during the implementation of the project.

Ubuntu 22.04 LTS

Ubuntu 22.04 LTS was used as the primary operating system for both the Host PC and the Industrial PC. Ubuntu provides a stable Linux environment that supports Python development, Docker containers, SSH communication, C++ compilation, and the Syncro5 Robot SDK.

The Linux environment offers excellent support for robotics applications because of its reliability, package management system, and compatibility with open-source development tools.

Features:

- Stable operating system
- Open-source and free to use
- Excellent support for Python and C++
- Easy installation of required packages
- Compatible with Docker and SSH
- Suitable for robotics development

Role in the Project:

- Running Python communication programs
- Compiling C++ robot programs
- Managing Docker containers
- Establishing SSH communication
- Operating the Syncro5 Robot SDK

Figure 13.1: Ubuntu 22.04 Desktop Environment
(Insert Ubuntu Desktop Screenshot here.)

Docker

Docker is a containerization platform that allows applications to run inside isolated environments called containers. The Syncro5 Robot SDK is designed to operate within a Docker container, ensuring that all required libraries, drivers, and dependencies remain properly configured.

Instead of installing robot libraries directly on the operating system, Docker provides a secure and consistent environment for robot control.

Features:

Container-based application deployment

Isolated execution environment

Dependency management

Easy portability

Improved system stability

Role in the Project:

Running the Syncro5 SDK

Executing robot control programs

Managing robot communication libraries

Maintaining a consistent development environment

Figure 13.2: Docker Architecture

(Insert Docker Communication Diagram here.)

Python

Python was used as the communication bridge between the ESP32 receiver and the collaborative robot. The Python program continuously reads serial

data from the receiver ESP32 and transfers the detected command to the Industrial PC using Secure Shell (SSH).

Python was selected because of its simplicity, extensive library support, and efficient handling of serial communication.

Libraries Used:

pySerial

subprocess

OpenCV (Gesture Control Module)

MediaPipe (Camera-based gesture recognition)

Role in the Project:

Reading ESP32 serial data

Processing gesture commands

Writing command files

Transferring commands through SSH

Camera-based gesture recognition

Figure 13.3: Python Communication Flow
(Insert Communication Flow Diagram here.)

C++

The robot control application was developed using C++ because the Syncro5 SDK provides its APIs in C++. C++ offers high execution speed, efficient memory management, and direct interaction with the robot controller.

The C++ program continuously monitors the command file and executes the corresponding robot movement or gripper operation.

Role in the Project:

Robot motion control
Gripper operation
Reading command file
Joint jogging
Robot SDK integration

Arduino IDE

Arduino IDE was used to develop and upload programs for both ESP32 microcontrollers.

The transmitter ESP32 processes flex sensor readings and generates gesture commands, while the receiver ESP32 receives wireless commands and forwards them to the Host PC through serial communication.

Role in the Project:

ESP32 programming
Program uploading
Serial debugging
Sensor testing

Figure 13.4: Arduino IDE
(Insert Arduino IDE Screenshot here.)

ESP32 Microcontroller

Two ESP32 development boards were used in this project.
The first ESP32 acts as the transmitter, processing flex sensor data and transmitting gesture commands wirelessly.

The second ESP32 acts as the receiver, receiving the commands and forwarding them to the Host PC through USB serial communication.

Features:

Built-in Wi-Fi
Dual-core processor
Low power consumption
High processing speed
ESP-NOW support

Role in the Project:

Gesture processing
Wireless communication
Command transmission
Serial communication

Figure 13.5: ESP32 Development Board
(Insert ESP32 Image here.)

ESP-NOW Protocol

ESP-NOW is a lightweight wireless communication protocol developed by Espressif. Unlike conventional Wi-Fi communication,

ESP-NOW allows two ESP32 devices to communicate directly without requiring a wireless router.

The protocol provides low communication latency, making it highly suitable for real-time gesture control.

Advantages:

Fast communication
Low latency
No Wi-Fi router required

Low power consumption
Reliable transmission

Role in the Project:

-Wireless communication between transmitter and receiver ESP32 modules.

SSH (Secure Shell)

SSH was used to securely transfer the command file from the Host PC to the Industrial PC.

The Python application automatically copies the updated command file whenever a new gesture is detected.

Role in the Project:

Secure communication
Remote file transfer
Communication between Host PC and Industrial PC

CMake

CMake is a build automation tool used to compile and link C++ applications.

It automatically generates Makefiles and manages dependencies required by the Syncro5 SDK.

Role in the Project:

Project configuration
Build management

Dependency linking

GCC / G++

The GNU Compiler Collection (GCC/G++) was used to compile all C++ source files into executable programs.

Role in the Project:

Compiling robot control programs
Building executable files
Error checking

Syncro5 Robot SDK

The Syncro5 Robot SDK provides APIs for controlling robot joints, gripper operations, safety functions, and communication with the robot controller. It forms the core software interface between the developed application and the collaborative robot.

Main Functions Used:

setupRobot()
doControl()
openGripper()
closeGripper()
reset()
getRobotData()

Role in the Project:

Robot initialization
Motion control
Gripper control
Controller management

SYSTEM ARCHITECTURE

The System Architecture describes the overall structure and working flow of the Smart Glove for Intelligent Device Control. It illustrates how different hardware and software components interact to perform real-time gesture-based control of intelligent devices. The architecture has been designed in a modular manner so that each component performs a specific function while communicating seamlessly with the others.

The complete system consists of a wearable smart glove equipped with flex sensors, an ESP32 transmitter, an ESP32 receiver, a Host Computer, an Industrial PC, Docker, and the Syncro5 Collaborative Robot. The flex sensors detect finger movements, which are converted into digital commands by the ESP32 transmitter. These commands are wirelessly transmitted to the receiver using the ESP-NOW protocol. The receiver forwards the commands to the Host Computer through USB serial communication.

The Host Computer processes the received commands using a Python application and securely transfers them to the Industrial PC using SSH. Inside the Docker container, a C++ application continuously monitors the command file and executes the corresponding robot movement or gripper operation through the Syncro5 Robot SDK. This modular architecture ensures reliable communication, low latency, and smooth robot operation.

Overall System Architecture

The complete architecture of the proposed system consists of the

following major modules:

- Smart Glove Module
- Flex Sensor Module
- ESP32 Transmitter Module
- ESP-NOW Wireless Communication
- ESP32 Receiver Module
- Host Computer
- Python Communication Program
- SSH Communication
- Industrial PC (Odyssey)
- Docker Environment
- C++ Robot Control Program
- Syncro5 Collaborative Robot

Working Flow of the System

The overall working of the proposed system follows the sequence shown below:

The user performs a predefined hand gesture while wearing the smart glove.

The flex sensors detect the finger bending and generate analog signals.

The ESP32 transmitter processes the sensor readings and identifies the corresponding gesture.

The detected gesture is converted into a control command.

The command is transmitted wirelessly to the ESP32 receiver using the ESP-NOW protocol.

The receiver sends the command to the Host Computer through USB serial communication.

The Python application reads the command and stores it in the command file.

The updated command file is securely transferred to the Industrial PC using SSH.

The C++ robot control program running inside the Docker container continuously monitors the command file.

The Syncro5 Robot SDK executes the corresponding robot movement or gripper operation.

The collaborative robot performs the desired action in real time.

Features of the System Architecture

Modular system design

Real-time gesture recognition

Wireless communication using ESP-NOW

Secure command transfer through SSH

Docker-based robot control environment

Real-time robot motion execution

Low communication latency

Easy scalability for future intelligent devices

Advantages of the Architecture:

Simple and modular communication flow

Easy maintenance and debugging

High reliability during operation

Fast response to user gestures

Secure communication between systems

Supports future hardware and software expansion

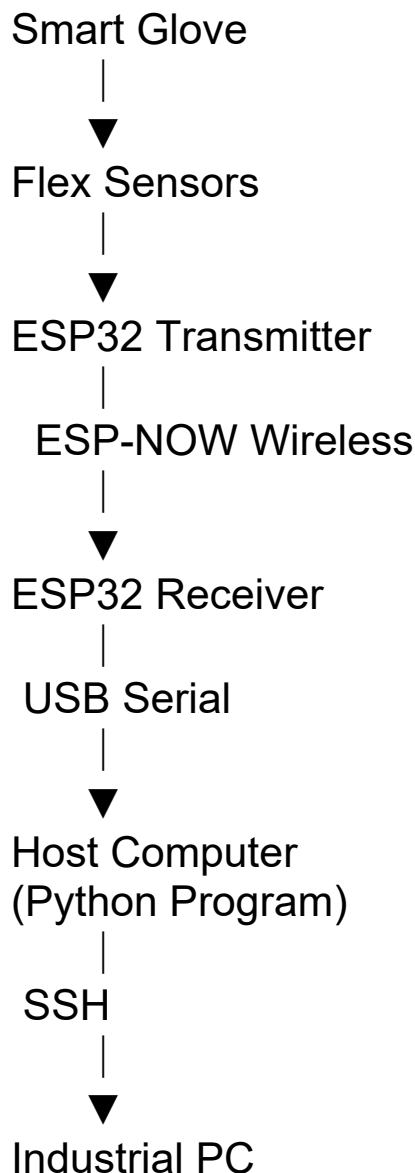
Role in the Project

The System Architecture acts as the backbone of the entire project. It defines the communication pathway from the user's hand gesture to the final

movement of the collaborative robot. The modular architecture simplifies integration, improves system reliability, and enables future expansion for controlling other intelligent devices such as RC cars, robotic arms, drones, wheelchairs, and IoT-based systems.

Overall System Architecture

Suggested diagram:



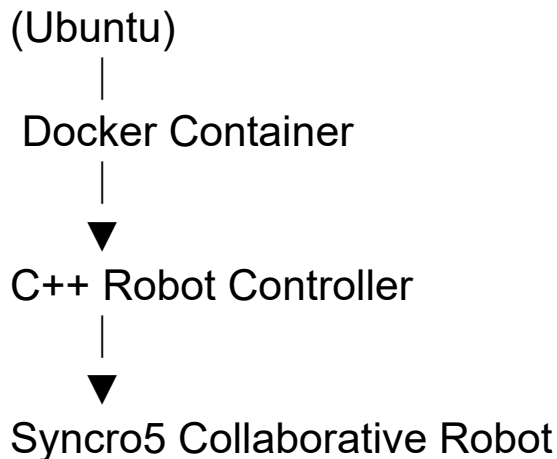


Figure 14.2: Communication Flow Between System Components
(Insert communication flow diagram showing data transfer between each module.)

SYSTEM DESIGN AND WORKING METHODOLOGY

The System Design and Working Methodology explains how the Smart Glove for Intelligent Device Control performs real-time control of the Syncro5 Collaborative Robot. The complete system has been designed by integrating hardware, embedded programming, wireless communication, Python scripting, SSH communication, Docker technology, and the Syncro5 Robot SDK.

The system follows a sequential communication process in which the user's hand gesture is converted into robot movement. Each module performs a specific task and passes the processed information to the next module until the robot executes the required action. This modular approach improves reliability, simplifies debugging, and allows future expansion for controlling other intelligent devices.

Smart Glove Design

The smart glove is the primary input device of the system. It was designed using a leather glove with a soft cotton inner lining to provide comfort during

prolonged use. Five flex sensors were mounted on the glove corresponding to the five fingers.

Each flex sensor detects the bending angle of its respective finger. The output of these sensors is continuously monitored by the ESP32 transmitter. A rechargeable lithium-ion battery powers the glove, making the system completely portable and wireless.

Features

Comfortable wearable design

Portable and lightweight

Battery powered

Real-time finger movement detection

Easy to wear and operate

Role in the Project

Captures user's hand gestures

Provides input to the control system

Enables natural human-robot interaction

Figure 15.1: Smart Glove Design

(Insert photograph of the developed smart glove.)

Flex Sensor Working

Each finger of the glove contains one 2.2-inch flex sensor. The electrical resistance of the sensor changes whenever the finger bends. The ESP32 continuously reads these analog values through its ADC pins.

Initially, the glove performs automatic calibration by recording the baseline value of each sensor while the hand is kept in a relaxed position. During operation, the current sensor values are compared with these baseline values to determine whether a finger is bent or straight.

To eliminate noise, an Exponential Moving Average (EMA) filter is applied before gesture recognition.

Features

Analog sensing

Automatic calibration

Noise filtering

High sensitivity

Real-time measurement

Role in the Project

Detects finger bending

Generates analog gesture information

Provides stable sensor readings

Figure 15.2: Flex Sensor Working Principle
(Insert flex sensor bending diagram.)

Gesture Recognition

The ESP32 transmitter continuously analyzes the status of all five fingers. Based on predefined gesture combinations, it identifies the intended robot command.

Each recognized gesture is converted into a single-character command before transmission.

Gesture Mapping

Hand Gesture	Command	Robot Action
Open Palm	x	Stop
Closed Fist	c	Close Gripper

Thumb Straight + Other Fingers Bent	o	Open Gripper
Thumb + Index Bent	w	+X Movement
Thumb + Middle Bent	s	-X Movement
Thumb + Ring Bent	a	+Y Movement
Thumb + Little Bent	d	-Y Movement
Thumb + Index + Middle Bent	u	+Z Movement
Thumb + Ring + Little Bent	n	-Z Movement

Features

Real-time gesture detection

Low processing time

Stable recognition

Simple command generation

Role in the Project

Converts hand gestures into robot commands

Reduces processing complexity

Improves communication speed

Figure 15.3: Gesture Mapping Chart

(Insert gesture photographs with corresponding commands.)

Wireless Communication Using ESP-NOW

After recognizing the gesture, the transmitter ESP32 sends the command to the receiver ESP32 using the ESP-NOW protocol.

ESP-NOW establishes direct communication between the two ESP32 boards without requiring a Wi-Fi router. This significantly reduces communication delay and improves reliability.

Features

Low latency

Fast communication

Low power consumption

No internet required

Role in the Project

Transfers gesture commands wirelessly

Maintains real-time communication

Ensures reliable data transfer

Figure 15.4: ESP-NOW Communication

(Insert ESP32 transmitter-receiver communication diagram.)

Serial Communication

The receiver ESP32 receives the wireless command and forwards it to the Host Computer through USB serial communication.

The receiver simply prints the received character to the serial port, allowing the Python application to capture it instantly.

Features

USB communication

High data reliability

Continuous transmission

Plug-and-play operation

Role in the Project

Transfers commands to the Host Computer

Acts as the interface between hardware and software

Figure 15.5: USB Serial Communication
(Insert receiver-to-PC connection diagram.)

Python Communication Module

The Python application continuously monitors the serial port using the PySerial library.

Whenever a new command is received, the program writes it into the command.txt file. To avoid unnecessary communication, the program only transfers the file when the command changes.

The updated command file is then securely copied to the Industrial PC using SSH.

Features

Real-time serial monitoring

Automatic command logging

File generation

SSH communication

Role in the Project

Reads serial data

Creates command file

Transfers commands to Industrial PC

Figure 15.6: Python Communication Process
(Insert Python workflow diagram.)

SSH Communication

SSH is used to establish secure communication between the Host Computer

and the Industrial PC.

Whenever a gesture changes, Python automatically copies the updated command file to the Industrial PC without requiring user intervention.

Features

Secure communication

Automatic file transfer

Reliable connection

Low latency

Role in the Project

Sends commands to the Industrial PC

Maintains secure communication

Figure 15.7: SSH Communication Flow

(Insert Host PC to Industrial PC communication diagram.)

Robot Control Module

Inside the Docker container, the C++ application continuously monitors the command file.

Whenever a new command is detected, the corresponding Syncro5 Robot SDK function is executed.

Examples include:

Joint jogging

Positive and negative axis movement

Opening gripper

Closing gripper

Emergency stop

Features

Continuous monitoring

Real-time execution

SDK integration

Fast response

Role in the Project

Executes robot movement

Controls gripper

Interfaces with Syncro5 SDK

Figure 15.8: Robot Control Flow

(Insert C++ command execution flowchart.)

Overall Working Sequence

The complete working process of the proposed system is summarized below:

User wears the smart glove.

Flex sensors detect finger movement.

ESP32 transmitter processes sensor data.

Gesture is recognized.

Command is generated.

Command is transmitted using ESP-NOW.

Receiver ESP32 receives the command.

Command is sent to the Host Computer via USB.

Python reads the serial data.

Python updates the command file.

SSH transfers the command file to the Industrial PC.

Docker-based C++ application reads the command.

Syncro5 SDK executes the corresponding robot action.

The robot performs the required movement.

HARDWARE IMPLEMENTATION

This chapter explains the hardware components used to develop the Smart Glove for Intelligent Device Control. The hardware section covers every device used in the project, including the wearable glove, sensors, microcontrollers, power supply, communication modules, collaborative robot, and supporting equipment. Each component plays a significant role in ensuring accurate gesture recognition, reliable communication, and smooth robot operation.

The hardware was selected based on availability, performance, ease of integration, low power consumption, and compatibility with the Syncro5 collaborative robot. Together, these components form a complete wearable robotic control system capable of controlling industrial devices through natural hand gestures.

Leather Smart Glove

A leather glove was used as the wearable platform for mounting all electronic components. The outer leather surface provides durability and protection, while the soft cotton inner lining offers comfort during extended use. The glove allows the flex sensors to move naturally with the fingers without affecting user comfort.

The glove acts as the Human-Machine Interface (HMI), allowing users to control the robot through simple hand movements instead of conventional controllers.

Features

- Comfortable to wear
- Durable leather material
- Soft cotton inner layer
- Lightweight design
- Easy sensor installation

Role in the Project

Base platform for mounting sensors

Captures user's hand movements

Provides wearable robot control interface

Figure 16.1: Smart Glove

(Insert photograph of the developed glove.)

Flex Sensors (2.2 Inch)

Five 2.2-inch flex sensors were mounted on the glove, one on each finger. A flex sensor changes its electrical resistance when bent. The ESP32 measures these resistance variations through its analog input pins and determines the finger position.

These sensors are the primary sensing elements of the project and are responsible for detecting all hand gestures used for robot control.

Features

Analog output

Flexible structure

Lightweight

High sensitivity

Easy interfacing with ESP32

Role in the Project

Detect finger bending

Generate analog gesture data

Enable gesture recognition

Figure 16.2: Flex Sensor

(Insert image of 2.2-inch flex sensor.)

ESP32 Development Boards

Two ESP32 development boards were used in the project.

The first ESP32 functions as the transmitter mounted on the glove. It continuously reads the flex sensor values, recognizes gestures, and sends commands wirelessly.

The second ESP32 acts as the receiver. It receives commands through ESP-NOW and forwards them to the Host PC using USB serial communication.

Features

Dual-core processor

Built-in Wi-Fi

ESP-NOW support

Low power consumption

Multiple ADC channels

Role in the Project

Gesture processing

Wireless communication

Command transmission

Serial communication

Figure 16.3: ESP32 Development Board
(Insert ESP32 image.)

10 k Ω Resistors

Each flex sensor was connected in a voltage divider circuit using a 10 k Ω resistor. This converts changes in sensor resistance into measurable voltage changes that the ESP32 can read accurately.

Features

Stable voltage divider
Low cost
Reliable operation

Role in the Project
Voltage divider circuit
Stable analog readings
Improved sensor accuracy

Figure 16.4: Flex Sensor Connection Circuit
(Insert voltage divider circuit diagram.)

Lithium-Ion Battery

The smart glove is powered using two rechargeable 3.7 V lithium-ion batteries. These batteries provide sufficient operating time while keeping the glove lightweight and portable.

Features
Rechargeable
High energy density
Lightweight
Portable

Role in the Project
Power supply for glove
Portable operation
Continuous wireless control

Figure 16.5: Lithium-Ion Battery
(Insert battery image.)

3S 12.6 V Charging Module

A 3S Type-C lithium battery charging module with boost converter was used to charge the battery pack safely. The module manages battery charging and provides regulated output voltage for the electronic components.

Features

- Type-C charging
- Overcharge protection
- Battery management
- Stable voltage output

Role in the Project

- Battery charging
- Voltage regulation
- Safe power management

Figure 16.6: Charging Module
(Insert charging module image.)

USB Cable

A USB cable connects the receiver ESP32 to the Host Computer. It supplies power to the receiver while also enabling serial communication between the receiver and the Python application.

Features

- Data transmission
- Power supply
- Plug-and-play

Role in the Project
USB serial communication
Receiver power supply

Figure 16.7: USB Connection
(Insert USB communication image.)

Industrial PC (Odyssey)

The Industrial PC serves as the robot controller. It runs Ubuntu Linux, Docker, and the Syncro5 SDK. The received commands are executed through the C++ robot control application inside the Docker container.

Features
Industrial-grade hardware
Ubuntu support
Docker compatibility
High processing capability

Role in the Project
Robot control
SDK execution
Docker environment
Motion control

Figure 16.8: Odyssey Industrial PC
(Insert Odyssey Industrial PC image.)

Host Computer

The Host Computer acts as the communication bridge between the receiver

ESP32 and the Industrial PC. It executes the Python communication program and transfers robot commands through SSH.

Features

- Python execution
- Serial communication
- SSH communication
- Command processing

Role in the Project

- Reads receiver data
- Generates command file
- Transfers commands to Industrial PC

Figure 16.9: Host Computer
(Insert Host PC photograph.)

Syncro5 Collaborative Robot

The Syncro5 Collaborative Robot is the main output device of the project. It receives commands from the C++ application and performs joint movements or gripper operations accordingly.

The robot consists of six rotational joints and a Feetech electric gripper, enabling it to perform industrial pick-and-place operations and other collaborative tasks.

Features

- Six Degrees of Freedom (6-DOF)
- High positional accuracy
- Collaborative operation
- Safety mechanisms
- SDK support

Role in the Project

- Executes gesture commands
- Performs robot movement
- Controls gripper operations

Figure 16.10: Syncro5 Collaborative Robot
(Insert photograph of Syncro5 robot.)

Feetech Electric Gripper

The Feetech gripper is mounted at the end effector of the robot. It enables the robot to grasp and release objects based on hand gestures detected by the smart glove.

Features

- Electric actuation
- High gripping accuracy
- SDK integration
- Reliable operation

Role in the Project

- Pick-and-place operations
- Object gripping
- Gesture-controlled manipulation

Figure 16.11: Feetech Gripper
(Insert image of Feetech gripper.)

COMMUNICATION

The Smart Glove for Intelligent Device Control consists of multiple hardware and software modules that communicate with each other to perform real-time

robot control. Each module has a specific responsibility, beginning with gesture detection and ending with the movement of the Syncro5 Collaborative Robot.

The communication system is designed to provide fast, secure, and reliable data transfer with minimum delay. Different communication methods such as analog sensing, ESP-NOW wireless communication, USB serial communication, SSH file transfer, and Docker-based robot control are integrated to complete the entire control process.

The communication flow of the project is divided into five major stages.

Glove → ESP32

The first stage begins with the smart glove. Five flex sensors mounted on the glove continuously detect the bending of each finger. The sensors generate analog voltage signals according to finger movement.

The ESP32 transmitter continuously reads these analog values through its ADC (Analog-to-Digital Converter) pins. Before gesture recognition, the sensor readings are filtered using an Exponential Moving Average (EMA) algorithm to reduce electrical noise and improve accuracy.

After filtering, the ESP32 compares the sensor values with the calibrated baseline values and determines the corresponding hand gesture.

Communication Type

Analog Signal

Data Transferred

Finger bending values

Gesture information

Role in the Project

Detect user hand movement

Convert analog sensor values into digital gestures

Generate robot control commands

Figure 17.1: Communication between Smart Glove and ESP32

(Insert Flex Sensor to ESP32 connection diagram.)

ESP₃₂ → ESP₃₂

After recognizing the gesture, the transmitter ESP32 converts it into a single-character command.

Examples include:

Gesture	Command
Stop	x
+X	w
-X	s
+Y	a
-Y	d
+Z	u
-Z	n
Open Gripper	o
Close Gripper	c

The command is transmitted wirelessly using the ESP-NOW protocol. ESP-NOW allows direct communication between the two ESP32 modules without requiring a Wi-Fi router, resulting in very low communication latency.

Communication Type
ESP-NOW Wireless Protocol

Data Transferred
Single-character robot command

Advantages
Low latency
Reliable communication
No router required
Low power consumption

Role in the Project
Wireless transmission of gesture commands
Fast real-time communication

Figure 17.2: ESP-NOW Communication
(Insert ESP32 Transmitter and Receiver communication diagram.)

Receiver → Host PC

The receiver ESP32 receives the wireless command and immediately forwards it to the Host Computer using USB serial communication.

The receiver does not process the received command. It simply prints the received character to the serial port, where the Python program continuously monitors incoming data.

Communication Type
USB Serial Communication

Data Transferred

Robot control command

Role in the Project

Transfer wireless command to the Host Computer

Provide real-time communication interface

Figure 17.3: Receiver ESP32 to Host Computer Communication
(Insert USB Serial communication diagram.)

Host PC → Docker

The Host Computer runs a Python application that continuously reads the serial port. Whenever a new command is received, the Python program updates the **command.txt** file and transfers it securely to the Industrial PC using SSH (Secure Shell).

The Docker container shares the robot network configuration directory, allowing the updated command file to become immediately available to the robot control application.

Communication Type

SSH File Transfer (SCP)

Data Transferred

Updated command.txt file

Software Used

Python

PySerial

subprocess

SSH

SCP

Role in the Project

Read serial data

Generate command file

Securely transfer robot commands

Figure 17.4: Host Computer to Industrial PC Communication
(Insert SSH communication diagram.)

Docker → Syncro5 Cobot

The final stage takes place inside the Docker container running on the Industrial PC. The C++ robot control program continuously monitors the **command.txt** file. Whenever the command changes, the program immediately calls the corresponding Syncro5 Robot SDK function.

Depending on the received command, the robot performs one of the following operations:

Joint movement

Positive axis movement

Negative axis movement

Open gripper

Close gripper

Stop robot

The SDK communicates directly with the Syncro5 robot controller and executes the required motion in real time.

Communication Type

Robot SDK API Communication

Data Transferred

Robot control instructions

Functions Used

setupRobot()

doControl()

openGripper()

closeGripper()

Role in the Project

Execute robot commands

Control robot joints

Operate the gripper

Perform real-time motion

Figure 17.5: Docker to Syncro5 Robot Communication

(Insert Docker → Robot control architecture.)

Overall Communication Flow

The complete communication sequence of the proposed system is illustrated below:

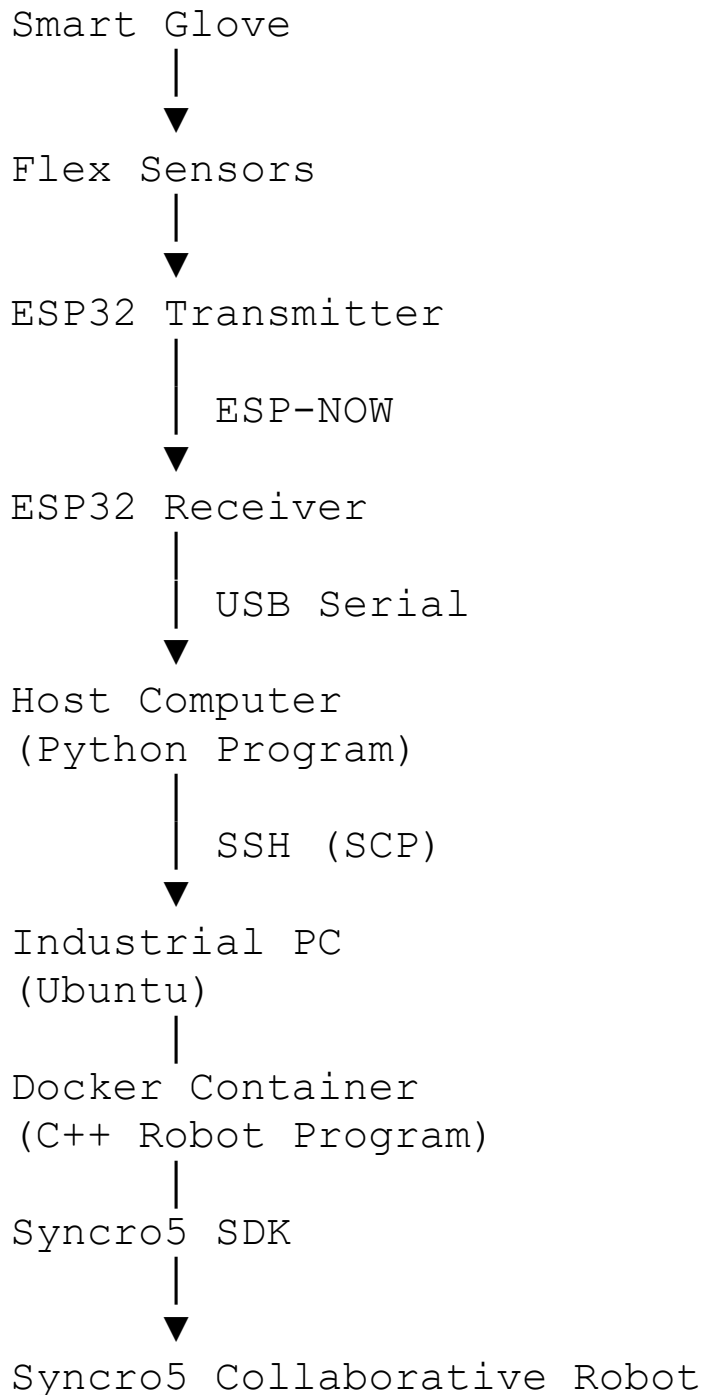


Figure 17.6: Complete Communication Flow of the Proposed System
(Insert the complete communication block diagram.)

HARDWARE DESCRIPTION

The hardware components used in the **Smart Glove for Intelligent Device Control** were selected to achieve accurate gesture recognition, reliable wireless communication, and real-time control of the Syncro5 Collaborative Robot. Each

component performs a specific function within the overall system and contributes to the efficient operation of the project.

The hardware setup consists of wearable sensing devices, microcontrollers, communication interfaces, power supply modules, and the collaborative robot. The integration of these components enables seamless interaction between the user and the robot through natural hand gestures.

Leather Smart Glove

The leather glove serves as the wearable platform for mounting all electronic components. It provides sufficient flexibility for natural finger movement while protecting the embedded sensors and wiring. The inner cotton lining ensures comfort during prolonged use.

The glove acts as the Human-Machine Interface (HMI), allowing users to control the robot through hand gestures instead of conventional controllers.

Specifications

Parameter	Description
Material	Leather (Outer), Cotton (Inner)
Type	Wearable Smart Glove
Number of Fingers	Five
Weight	Lightweight
Purpose	Gesture Detection Platform

Functions

Holds all flex sensors

Supports ESP32 transmitter

Allows natural hand movement

Provides wearable interface

Figure 18.1

Leather Smart Glove

(Insert actual glove photograph.)

Flex Sensors (2.2 Inch)

Five 2.2-inch flex sensors are mounted on the glove, one for each finger. A flex sensor changes its resistance when bent, allowing the ESP32 to detect finger movements accurately.

The sensor readings are processed to identify predefined hand gestures used for robot control.

Specifications

Parameter	Description
Sensor Type	Flex Sensor
Length	2.2 Inch
Quantity	5

Output	Analog Resistance
Operating Voltage	3.3V

Functions

Detect finger bending

Generate analog signals

Enable gesture recognition

Figure 18.2

2.2 Inch Flex Sensor

(Insert flex sensor image.)

ESP32 Development Board (Transmitter)

The transmitter ESP32 is mounted directly on the glove. It continuously reads the analog values of the flex sensors, recognizes gestures, and transmits the generated commands wirelessly using ESP-NOW.

Specifications

Parameter	Description
Controller	ESP32
Clock Speed	240 MHz
Wi-Fi	Yes
Bluetooth	Yes
ADC Channels	Multiple
Operating Voltage	3.3V

Functions

Read flex sensors
Detect gestures
Generate commands
Wireless transmission

Figure 18.3

ESP32 Transmitter

(Insert transmitter photograph.)

ESP32 Development Board (Receiver)

The receiver ESP32 receives commands from the transmitter and forwards them to the Host Computer through USB serial communication.

Specifications

Parameter	Description
Controller	ESP32
Communication	ESP-NOW + USB Serial
Operating Voltage	USB Powered

Functions

- Receive wireless commands
- Serial communication
- Forward commands to Python program

Figure 18.4

ESP32 Receiver

(Insert receiver photograph.)

10 kΩ Resistors

Each flex sensor is connected using a 10 kΩ resistor to form a voltage divider circuit. This arrangement converts resistance changes into measurable voltage variations.

Specifications

Parameter	Description
Resistance	10 kΩ
Quantity	5
Type	Carbon Film

Functions

Voltage divider

Stable analog readings

Improve sensor accuracy

Figure 18.5

Voltage Divider Circuit

(Insert circuit diagram.)

Lithium-Ion Battery Pack

The smart glove is powered by two rechargeable 3.7 V lithium-ion batteries connected with a charging module. The battery pack provides sufficient power for the ESP32 transmitter and flex sensor circuit.

Specifications

Parameter	Description
Battery Type	Lithium-Ion

Voltage	3.7 V × 2
Rechargeable	Yes

Functions

Portable power source

Continuous glove operation

Figure 18.6

Lithium-Ion Battery

(Insert battery image.)

3S Type-C Charging Module

A 3S 12.6 V charging module with boost converter is used to charge the battery pack safely. It provides battery protection and regulated output voltage.

Specifications

Parameter	Description
Charging Type	Type-C
Output Voltage	12.6 V
Current Rating	4 A

Functions

Battery charging

Voltage regulation

Overcharge protection

Figure 18.7

Battery Charging Module

(Insert charging module image.)

USB Cable

A USB cable connects the receiver ESP32 to the Host Computer. It provides both power and serial communication.

Specifications

Parameter	Description
Interface	USB
Function	Power + Data

Functions

Serial communication

Power supply

Figure 18.8

USB Connection

(Insert USB connection photograph.)

Host Computer

The Host Computer executes the Python communication program. It receives serial data from the ESP32 receiver and securely transfers robot commands to the Industrial PC.

Specifications

Parameter	Description
Operating System	Ubuntu 22.04 LTS

Programming	Python 3.10
Communication	SSH, USB Serial

Functions

Execute Python program

Process commands

SSH communication

Figure 18.9

Host Computer

(Insert Host PC photograph.)

Industrial PC (Odyssey)

The Industrial PC hosts the Docker container and executes the Syncro5 Robot SDK. It receives command files from the Host Computer and controls the robot through the C++ application.

Specifications

Parameter	Description
System	Odyssey Industrial PC
Operating System	Ubuntu 22.04
Environment	Docker

Functions

Execute Docker container

Run C++ robot controller

Interface with robot SDK

Figure 18.10

Odyssey Industrial PC
(Insert Industrial PC image.)

Syncro5 Collaborative Robot

The Syncro5 Collaborative Robot is the output device of the system. It executes the received commands by moving its six joints and operating the Feetech gripper.

Specifications

Parameter	Description
Robot Type	Collaborative Robot
Degrees of Freedom	6 DOF
End Effector	Feetech Electric Gripper

Functions

Joint movement

Pick-and-place operation

Gesture-based control

Figure 18.11

Syncro5 Collaborative Robot

(Insert robot photograph.)

Feetech Electric Gripper

The Feetech gripper is mounted on the end effector of the robot. It performs gripping and releasing operations according to the received gesture commands.

Specifications

Parameter	Description

Type	Electric Gripper
Control	SDK Controlled
Mounting	Robot End Effector

Functions

Pick objects

Release objects

Perform manipulation tasks

Figure 18.12

Feetech Electric Gripper

(Insert gripper image.)

SOFTWARE DESCRIPTION

The development of the Smart Glove for Intelligent Device Control required the integration of several software platforms, programming languages, libraries, and development tools. Each software component played a specific role, ranging from embedded programming and wireless communication to robot control and application development.

The software architecture was designed to ensure reliable communication between the smart glove and the collaborative robot while maintaining real-time performance. Different software tools were used for programming the ESP32 microcontrollers, processing sensor data, transferring commands, compiling robot applications, and controlling the Syncro5 Collaborative Robot.

The following software platforms were used during the development of this project.

Ubuntu 22.04 LTS

Ubuntu 22.04 LTS served as the primary operating system for both the Host PC and the Industrial PC. It provided a stable Linux environment for software development, communication, Docker management, and robot control. Ubuntu was selected because of its excellent compatibility with robotics frameworks, open-source development tools, and industrial automation software.

Features

- Open-source operating system
- Stable Linux platform

Long-Term Support (LTS)

- Excellent package management
- Compatible with robotics software

Role in the Project

- Runs Python applications
- Compiles C++ robot programs
- Hosts Docker containers
- Supports SSH communication
- Executes the Syncro5 SDK

Figure 19.1: Ubuntu 22.04 Desktop
(Insert Ubuntu desktop screenshot.)

Arduino IDE

Arduino IDE was used for programming and uploading firmware to both ESP32 development boards. The transmitter program reads flex sensor values and recognizes gestures, while the receiver program receives wireless commands and forwards them to the Host PC through USB serial communication.

Features

- Easy code editing
- ESP32 board support
- Serial Monitor
- Built-in compiler
- One-click firmware upload

Role in the Project

- Programming ESP32 transmitter
- Programming ESP32 receiver
- Sensor testing
- Serial debugging

Figure 19.2: Arduino IDE
(Insert Arduino IDE screenshot.)

Python 3.10

Python was used as the communication bridge between the receiver ESP32 and the Industrial PC. The Python application continuously monitors the serial port, stores received commands into a command file, and securely transfers the file to the Industrial PC using SSH. Python was also used during the development of the camera-based gesture control system using OpenCV and MediaPipe.

Features

- Easy syntax

- Fast development
- Large standard library
- Cross-platform support
- Excellent serial communication support
- Major Libraries Used
- pySerial
- subprocess
- OpenCV
- MediaPipe

Role in the Project

- Serial communication
- Command processing
- SSH file transfer

Figure 19.3: Python Development Environment
(Insert Python IDE screenshot.)

C++

The robot control software was developed using C++ because the Syncro5 SDK provides its APIs in C++. The C++ application continuously monitors the command file and executes robot movements or gripper operations according to the received commands.

Features

- High execution speed
- Efficient memory management
- Object-oriented programming
- Real-time performance

Role in the Project

- Robot motion control
- Gripper control
- Command processing
- SDK integration

Figure 19.4: C++ Development
(Insert VS Code or terminal compilation screenshot.)

Docker

Docker was used to execute the Syncro5 Robot SDK in an isolated environment. Instead of installing robot libraries directly on Ubuntu, the SDK runs inside a Docker container containing all required dependencies.

This approach improves system stability and ensures consistent robot operation.

Features

- Containerization
- Dependency isolation
- Easy deployment
- Consistent execution
- Secure environment

Role in the Project

- Running Syncro5 SDK
- Executing robot applications
- Managing software dependencies

Figure 19.5: Docker Container

(Insert Docker architecture or terminal screenshot.)

CMake

CMake was used as the build management system for compiling the C++ robot control applications. It automatically generated Makefiles and linked all necessary robot libraries required by the Syncro5 SDK.

Features

- Automatic build generation
- Dependency management
- Cross-platform support
- Simple project configuration

Role in the Project

- Configure robot projects
- Generate Makefiles
- Manage SDK libraries

Figure 19.6: CMake Build Process

(Insert Cmake build screenshot.)

GCC / G++

The GNU Compiler Collection (GCC/G++) was used to compile all C++ source files into executable robot control programs.

It also helped identify syntax errors and ensured successful integration with the Syncro5 SDK libraries.

Features

- Fast compilation
- Error detection
- Standards-compliant compiler
- Optimized executable generation

Role in the Project

- Compile C++ applications
- Generate executable files
- Debug compilation errors

Figure 19.7: GCC Compilation

(Insert compilation terminal screenshot.)

SSH (Secure Shell)

SSH was used to establish secure communication between the Host PC and the Industrial PC.

Whenever the Python application detected a new gesture command, it automatically transferred the updated `command.txt` file to the Industrial PC using the SCP utility.

Features

- Secure communication
- File transfer
- Remote system access
- Encrypted data transmission

Role in the Project

- Secure command transfer
- Remote communication
- Industrial PC connectivity

Figure 19.8: SSH Communication

(Insert SSH terminal screenshot.)

Syncro5 Robot SDK

The Syncro5 Robot SDK is the primary software framework used to control the collaborative robot.

It provides a collection of APIs for robot initialization, controller selection, joint movement, Cartesian movement, gripper control, recording mode, playback mode, and safety management.

The SDK acts as the interface between the developed C++ application and the robot controller.

Major Functions Used

- `setupRobot()`
- `doControl()`
- `reset()`
- `getRobotData()`
- `openGripper()`
- `closeGripper()`

Features

- Complete robot control APIs
- Motion planning

- Safety management
- Gripper control
- Controller switching

Role in the Project

- Robot initialization
- Motion execution
- Gripper operation
- Communication with the robot controller

Figure 19.9: Syncro5 SDK Architecture

(Insert SDK architecture or project directory screenshot.)

PROGRAM DEVELOPMENT

The software development of the Smart Glove for Intelligent Device Control was carried out in multiple stages. Instead of developing a single application, the project was divided into four independent software modules. Each module performs a specific task and communicates with the next module to achieve real-time robot control. The modular programming approach simplified debugging, testing, maintenance, and future expansion of the system. The four major software modules developed during this project are:

ESP32 Transmitter Program

ESP32 Receiver Program

Python Communication Program

C++ Robot Controller Program

Each module is explained in detail below.

ESP32 Transmitter

The ESP32 transmitter program is responsible for acquiring hand gesture information from the smart glove. Five flex sensors mounted on the glove continuously provide analog values corresponding to the bending of each finger.

The program first performs an automatic calibration process by recording the initial sensor values when the user's hand is in the open position. These baseline values are later used to determine whether each finger is bent or straight.

To improve measurement stability, an Exponential Moving Average (EMA) filter is applied to the sensor readings. The filtered values are compared with predefined threshold margins, and the corresponding gesture is identified. After successful gesture recognition, the program converts the gesture into a single-character command and transmits it to the receiver ESP32 using the ESP-NOW wireless communication protocol.

Main Functions

- Read flex sensor values
- Automatic sensor calibration
- Noise reduction using EMA filter
- Gesture recognition
- Command generation
- ESP-NOW wireless transmission

Programming Logic

- Initialize ESP32 and ESP-NOW.
- Read analog values from all five flex sensors.
- Apply EMA filtering to remove noise.
- Compare values with calibrated baselines.
- Identify the corresponding hand gesture.
- Generate the command character.
- Transmit the command using ESP-NOW.
- Repeat continuously.

Commands Generated

Gesture	Command
Stop	x
+X	w
-X	s
+Y	a
-Y	d
+Z	u
-Z	n
Open Gripper	o
Close Gripper	c

Figure 20.1

ESP32 Transmitter Program Flowchart

//glove transmitter program.

#include <esp_now.h>

#include <WiFi.h>

// --- Pin Definitions ---

const int THUMB_PIN = 32;

const int INDEX_PIN = 33;

const int MIDDLE_PIN = 34;

const int RING_PIN = 35;

const int LITTLE_PIN = 39;

// --- Trigger Margins ---

const int MARGIN_T = 8;

const int MARGIN_I = 12;

const int MARGIN_M = 22;

const int MARGIN_R = 22;

const int MARGIN_L = 22;

// --- Baseline Variables ---

int baseT = 0, baseI = 0, baseM = 0, baseR = 0, baseL = 0;

// --- EMA Smoothing Variables (The "Shock Absorber") ---

float emaT = 0, emaI = 0, emaM = 0, emaR = 0, emaL = 0;

float smoothing = 0.1; // 10% New Reading, 90% Old Reading

// --- Lockout Timer Variables (The "Security Guard") ---

char active_command = 'x'; // The currently locked-in command

char pending_command = 'x'; // The command we are evaluating

unsigned long cmd_start_time = 0;

const unsigned long HOLD_TIME = 500; // 500ms to lock a command

// RECEIVER'S MAC ADDRESS

uint8_t broadcastAddress[] = {0xC8, 0x2E, 0x18, 0x25, 0xB1, 0xC8};

typedef struct struct_message {

char command;

} struct_message;

struct_message myData;

esp_now_peer_info_t peerInfo;

// --- The EMA Math Filter ---

void getSmoothedReadings(int &t, int &i, int &m, int &r, int &l) {

```

// Blend the new raw reading with the heavy old reading
emaT = (emaT * (1.0 - smoothing)) + (analogRead(THUMB_PIN) * smoothing);
emaI = (emaI * (1.0 - smoothing)) + (analogRead(INDEX_PIN) * smoothing);
emaM = (emaM * (1.0 - smoothing)) + (analogRead(MIDDLE_PIN) * smoothing);
emaR = (emaR * (1.0 - smoothing)) + (analogRead(RING_PIN) * smoothing);
emaL = (emaL * (1.0 - smoothing)) + (analogRead(LITTLE_PIN) * smoothing);

// Convert the smoothed decimal math back into whole numbers
t = (int)emaT;
i = (int)emaI;
m = (int)emaM;
r = (int)emaR;
l = (int)emaL;
}

void setup() {
  Serial.begin(115200);
  WiFi.mode(WIFI_STA);

  if (esp_now_init() != ESP_OK) {
    Serial.println("Error initializing ESP-NOW");
    return;
  }

  memcpy(peerInfo.peer_addr, broadcastAddress, 6);
  peerInfo.channel = 0;
  peerInfo.encrypt = false;

  if (esp_now_add_peer(&peerInfo) != ESP_OK){
    Serial.println("Failed to add peer");
    return;
  }

  // "Prime" the math filter so it doesn't start at zero
  emaT = analogRead(THUMB_PIN);
  emaI = analogRead(INDEX_PIN);
  emaM = analogRead(MIDDLE_PIN);
  emaR = analogRead(RING_PIN);
  emaL = analogRead(LITTLE_PIN);

  // --- AUTO CALIBRATION PHASE ---
  Serial.println("CALIBRATING: KEEP HAND COMPLETELY FLAT!");
  delay(3000); // Give user 3 seconds to flatten hand

```

```

// Lock in the resting baselines
getSmoothedReadings(baseT, baseI, baseM, baseR, baseL);

Serial.println("Baselines Locked! Ready to control cobot.");
}

void loop() {
  int currT, currI, currM, currR, currL;

  // Get the heavily smoothed, noise-free numbers
  getSmoothedReadings(currT, currI, currM, currR, currL);

  // Determine if the finger has deliberately moved past the margin
  bool t = abs(currT - baseT) > MARGIN_T;
  bool i = abs(currI - baseI) > MARGIN_I;
  bool m = abs(currM - baseM) > MARGIN_M;
  bool r = abs(currR - baseR) > MARGIN_R;
  bool l = abs(currL - baseL) > MARGIN_L;

  char raw_cmd = 'x'; // Default to STOP

  // --- Gesture Logic Mapping ---
  if (!t && !i && !m && !r && !l) {
    raw_cmd = 'x'; // Open palm -> Stop
  }
  else if (t && i && m && r && l) {
    raw_cmd = 'c'; // Full fist -> Close gripper
  }
  else if (!t && i && m && r && l) {
    raw_cmd = 'o'; // Four fingers bent, thumb straight -> Open gripper
  }
  else if (t && i && !m && !r && !l) {
    raw_cmd = 'w'; // Thumb + Index bent -> +X
  }
  else if (t && !i && m && !r && !l) {
    raw_cmd = 's'; // Thumb + Middle bent -> -X
  }
  else if (t && !i && !m && r && !l) {
    raw_cmd = 'a'; // Thumb + Ring bent -> +Y
  }
  else if (t && !i && !m && !r && l) {
    raw_cmd = 'd'; // Thumb + Little bent -> -Y
  }
  else if (t && i && m && !r && !l) {

```

```

    raw_cmd = 'u'; // Thumb + Index + Middle bent -> +Z
}
else if (t && !i && !m && r && l) {
    raw_cmd = 'n'; // Thumb + Ring + Little bent -> -Z
}

// --- STRICT COMMAND LOCKOUT LOGIC ---

if (active_command != 'x') {
    // 1. WE ARE CURRENTLY MOVING (LOCKED STATE)
    // We completely ignore all twitches and cross-talk. We ONLY look for an open hand ('x') to stop.
    if (raw_cmd == 'x') {
        // Hand opened! Break the lock and stop instantly.
        active_command = 'x';
        pending_command = 'x';
        myData.command = 'x';
        esp_now_send(broadcastAddress, (uint8_t *) &myData, sizeof(myData));
        Serial.println(">>> STOPPING & UNLOCKING <<<");
    } else {
        // User is still holding gesture (or twitching). Keep sending the locked command.
        myData.command = active_command;
        esp_now_send(broadcastAddress, (uint8_t *) &myData, sizeof(myData));
    }
}
else {
    // 2. WE ARE IDLE (WAITING FOR A NEW GESTURE)
    if (raw_cmd == 'x') {
        // Hand is flat. Stay idle.
        pending_command = 'x';
    }
    else {
        // Attempting a new gesture!
        if (raw_cmd != pending_command) {
            // Just started the gesture. Start the stopwatch.
            pending_command = raw_cmd;
            cmd_start_time = millis();
        }
        else {
            // Holding the gesture. Has half a second passed?
            if (millis() - cmd_start_time >= HOLD_TIME) {
                // TIME REACHED! LOCK IT IN!
                active_command = raw_cmd;
                myData.command = active_command;
                esp_now_send(broadcastAddress, (uint8_t *) &myData, sizeof(myData));
            }
        }
    }
}

```

```

        Serial.print(">>> LOCKED IN COMMAND: ");
        Serial.println(active_command);
    }
}
}
}

// A tiny 10ms delay to prevent flooding the receiver
delay(10);
}

```

ESP32 Receiver

The receiver program continuously listens for incoming ESP-NOW packets from the transmitter.

Whenever a packet is received, the receiver extracts the command character and immediately prints it to the USB serial port. No additional processing is performed by the receiver, keeping the communication fast and efficient. The Host Computer reads this serial data using the Python communication program.

Main Functions

- Initialize ESP-NOW receiver
- Receive wireless commands
- Decode incoming packets
- Send command through USB serial communication

Programming Logic

- Initialize ESP-NOW receiver.
- Wait for incoming packet.
- Receive command.
- Print command to Serial Monitor.
- Wait for next command.

ESP32 Receiver Program Flowchart

```
//reciever program.
#include <esp_now.h>
#include <WiFi.h>

typedef struct struct_message {
    char command;
} struct_message;

struct_message myData;

// --- UPDATED CALLBACK FOR ESP32 CORE 3.x ---
void OnDataRecv(const esp_now_recv_info_t *info, const uint8_t *incomingData, int len) {
    memcpy(&myData, incomingData, sizeof(myData));

    // Print ONLY the character.
    // Your C++ program will read this stream.
    Serial.println(myData.command);
}

void setup() {
    Serial.begin(115200);
    WiFi.mode(WIFI_STA);

    if (esp_now_init() != ESP_OK) {
        Serial.println("Error initializing ESP-NOW");
        return;
    }

    // Updated registration function
    esp_now_register_recv_cb(OnDataRecv);
}

void loop() {
    // Everything happens automatically in the callback
}
```

Python Communication

The Python communication program serves as the communication bridge between the receiver ESP32 and the Industrial PC.

The program continuously monitors the serial port using the PySerial library. Whenever a new command is received, it writes the command into a command.txt file. To minimize unnecessary network traffic, the file is transferred only when the received command changes.

The updated command file is securely copied to the Industrial PC using SSH (SCP), where it becomes available to the robot control application running inside the Docker container.

Main Functions

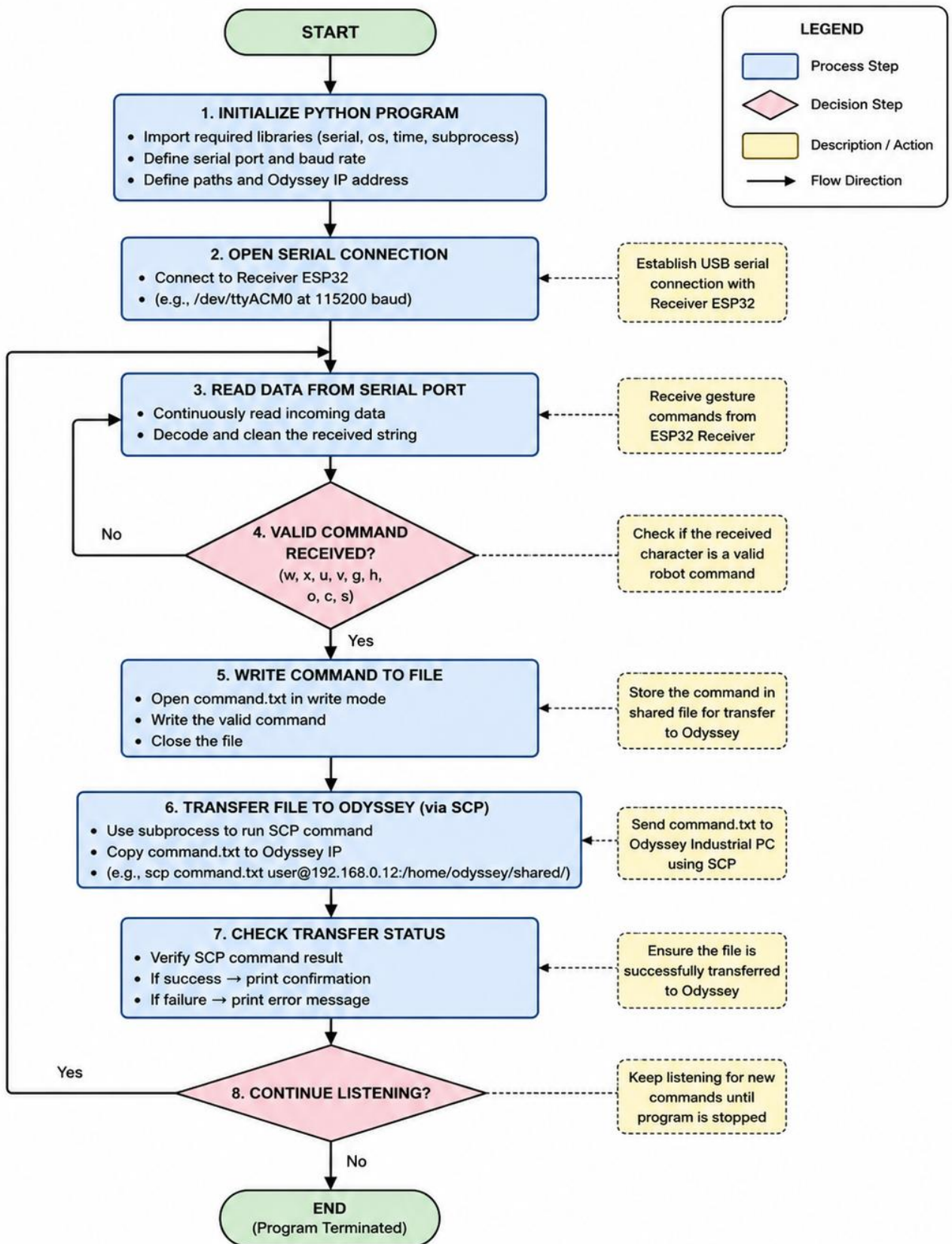
- Monitor serial port
- Read receiver commands
- Generate command file
- Detect command changes
- Transfer file using SSH

Programming Logic

- Open serial communication.
- Wait for incoming command.
- Read received character.
- Compare with previous command.
- Update command.txt.
- Transfer file using SCP.
- Repeat continuously.
- Libraries Used
 - pySerial
 - subprocess

Figure 20.3

Python Communication Program Flowchart



C++ Robot Controller

The C++ Robot Controller is the final software module responsible for controlling the Syncro5 Collaborative Robot. The application is executed inside the Docker container and continuously monitors the command.txt file. Whenever a new command is detected, it calls the corresponding Syncro5 SDK function to execute the required robot motion. Depending on the received command, the program performs joint jogging, positive or negative axis movement, gripper opening, gripper closing, or robot stop operations. The robot controller continuously executes the control loop, ensuring smooth and responsive robot motion.

Main Functions

- Robot initialization
- SDK communication
- Read command file
- Joint jogging
- Gripper control
- Continuous robot control

Programming Logic

- Initialize Syncro5 Robot SDK.
- Connect to the robot.
- Open command.txt.
- Read latest command.
- Execute corresponding robot function.
- Repeat continuously.

SDK Functions Used

- setupRobot()
- doControl()
- openGripper()
- closeGripper()
- reset()
- getRobotData()

Figure 20.4

Robot Controller Flowchart

(Insert C++ program flowchart.)

Integrated Software Workflow

The four software modules work together to create a complete real-time control system. Each module performs an independent task while communicating with the next stage of the system.

The complete software workflow is as follows:

1. ESP32 Transmitter detects hand gestures.
2. Commands are transmitted wirelessly through ESP-NOW.

3. ESP32 Receiver forwards commands via USB serial communication.
4. Python program receives the commands and updates the command file.
5. SSH transfers the updated command file to the Industrial PC.
6. Docker-based C++ program reads the command file.
7. Syncro5 Robot SDK executes the corresponding robot action.

Figure 20.5

Complete Software Workflow

Program Development Summary

The software for the Smart Glove for Intelligent Device Control was developed using a modular programming approach to improve reliability, maintainability, and scalability. Each module was independently designed, tested, and integrated into the overall system.

The ESP32 transmitter handled gesture acquisition, the receiver managed wireless data reception, the Python application acted as the communication bridge, and the C++ robot controller executed the final robot actions through the Syncro5 SDK.

This architecture enabled efficient real-time operation and provides a strong foundation for future enhancements, including the control of additional intelligent devices such as RC cars, robotic arms, drones, and IoT-based systems.

Gesture Recognition and Command Mapping

Gesture recognition is the core functionality of the Smart Glove for Intelligent Device Control. It enables the system to interpret the user's hand movements and convert them into meaningful commands for controlling the Syncro5 Collaborative Robot. The objective of the gesture recognition module is to provide an intuitive and natural method of robot interaction without using conventional controllers such as joysticks or keyboards.

The recognition process begins by continuously monitoring the five flex sensors attached to the glove. Each sensor measures the bending of a specific finger and sends analog data to the ESP32 transmitter. After signal filtering and processing, the system determines the current finger positions and compares them with predefined gesture patterns. Once a valid gesture is detected, it is converted into a unique command character and transmitted to the robot control system.

Gesture Recognition Process

1. The gesture recognition process consists of several sequential stages. Each stage contributes to accurate and reliable gesture detection.
2. Read analog values from all five flex sensors.
3. Apply Exponential Moving Average (EMA) filtering to reduce noise.
4. Compare sensor values with calibrated baseline values.
5. Determine whether each finger is bent or straight.
6. Match the finger combination with predefined gesture patterns.
7. Generate the corresponding command.
8. Transmit the command to the receiver ESP32 using ESP-NOW.
9. This process is continuously repeated, allowing the robot to respond to the user's hand movements in real time.

Figure 21.1



Gesture Recognition Process

Finger State Detection

Each flex sensor is assigned to one finger of the hand. During operation, the ESP32 continuously evaluates the state of every finger.

If the sensor value exceeds the predefined threshold, the finger is considered Bent. Otherwise, it is considered Straight.

The combination of these five finger states forms a unique gesture that is recognized by the system.

Finger Assignment

Finger	Flex Sensor

Thumb	Sensor 1
Index Finger	Sensor 2
Middle Finger	Sensor 3
Ring Finger	Sensor 4
Little Finger	Sensor 5

Figure 21.2
Finger Assignment on Smart Glove

Command Mapping Strategy

After identifying the current finger positions, the system compares the detected pattern with predefined gesture combinations.

Each gesture is mapped to a unique command character. These commands are intentionally kept as single characters to minimize communication delay and simplify processing by the robot controller.

This mapping ensures fast and reliable communication between the glove and the collaborative robot.

Gesture Command Table

The gesture mapping implemented in this project is shown below.

Thumb	Index	Middle	Ring	Little	Command	Robot Action
Straight	Straight	Straight	Straight	Straight	x	Stop Robot
Bent	Bent	Bent	Bent	Bent	c	Close Gripper
Straight	Bent	Bent	Bent	Bent	o	Open Gripper
Bent	Bent	Straight	Straight	Straight	w	Move +X
Bent	Straight	Bent	Straight	Straight	s	Move -X
Bent	Straight	Straight	Bent	Straight	a	Move +Y
Bent	Straight	Straight	Straight	Bent	d	Move -Y
Bent	Bent	Bent	Straight	Straight	u	Move +Z
Bent	Straight	Straight	Bent	Bent	n	Move -Z

Table 21.1: Gesture Command Mapping

Robot Motion Mapping

Each generated command corresponds to a predefined robot action. After the command reaches the robot controller, the appropriate SDK function is executed.

Command	Robot Motion
x	Stop all joint motion
w	Joint movement in +X direction
s	Joint movement in -X direction
a	Joint movement in +Y direction
d	Joint movement in -Y direction
u	Joint movement in +Z direction
n	Joint movement in -Z direction
o	Open gripper
c	Close gripper

Table 21.2: Robot Motion Mapping

Noise Reduction Technique

Sensor noise and small unintended finger movements can lead to incorrect gesture recognition. To improve stability, the system applies an Exponential Moving Average (EMA) filter to every sensor reading. The filter smooths sudden fluctuations while maintaining fast response to intentional gestures. As a result, the generated commands remain stable and consistent during operation.

In addition to EMA filtering, automatic calibration is performed during system startup. The glove records the initial finger positions and uses these values as reference points for subsequent gesture recognition.

Advantages

- Reduces electrical noise
 - Improves recognition accuracy
 - Minimizes false gesture detection
 - Provides stable command generation
-

Command Validation

To prevent accidental robot movement, the software validates every detected gesture before transmitting it.

The system only sends a new command when:

- A valid gesture pattern is detected.
 - The detected command differs from the previously transmitted command.
 - The gesture remains stable after signal filtering.
 - This validation process minimizes unnecessary communication and improves overall system reliability.
-

Gesture Recognition Workflow

The complete workflow followed by the gesture recognition module is summarized below.

- User bends fingers.
- Flex sensors detect finger positions.
- ESP32 reads analog values.
- Sensor data is filtered using EMA.
- Finger states are determined.
- Gesture pattern is recognized.
- Command character is generated.
- Command is transmitted using ESP-NOW.
- Robot receives and executes the corresponding action.

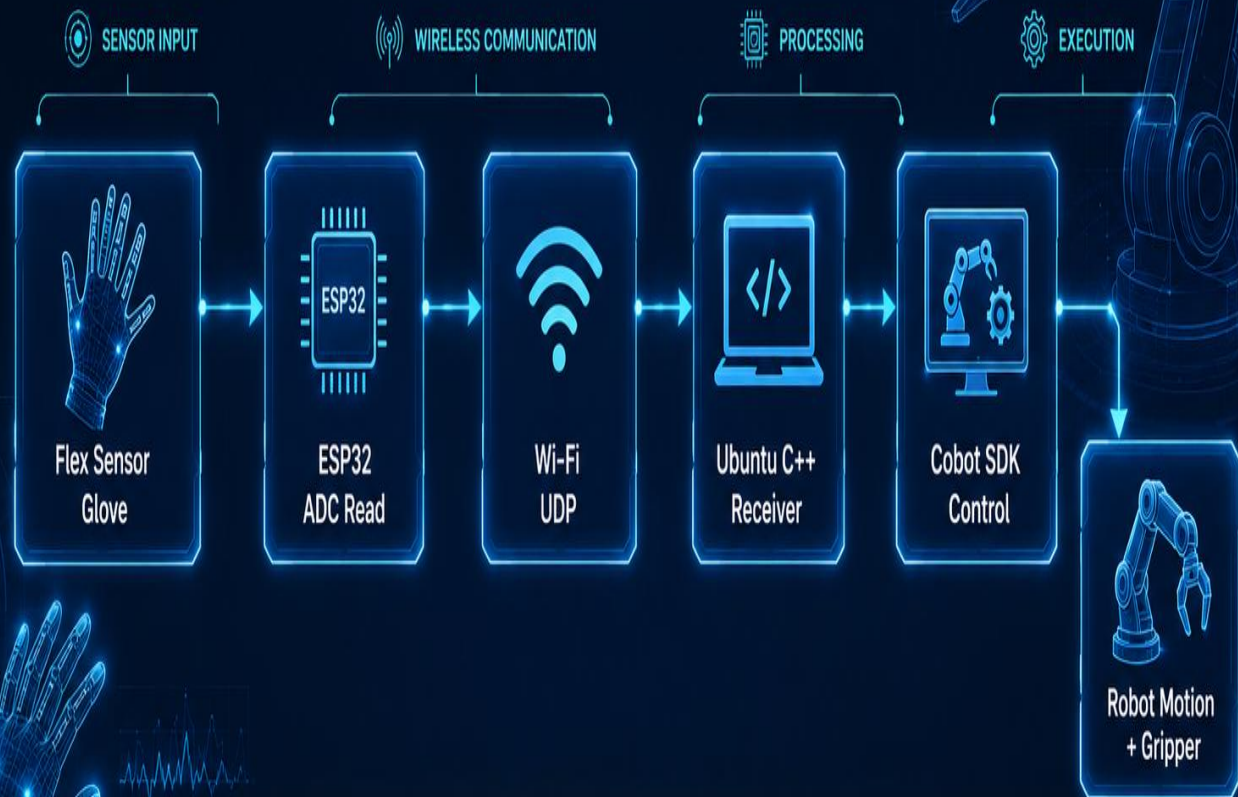
Figure 21.3

Gesture Recognition Workflow


```
Activities Terminator Jul 20 10:01
cobot@cobot-HP-ProDesk-2-Tower-G1a-Desktop-AI-PC: ~
cobot@cobot-HP-ProDesk-2-Tower-G1a-Desktop-AI-PC: ~ 122x33
cobot@cobot-HP-ProDesk-2-Tower-G1a-Desktop-AI-PC:~$ ls
'ADVERB TEAM MAIL ID' Documents install Templates
Arduino Downloads log 'Untitled 1.odt'
build dual_cam_pick Music Videos
cap_%d 'FOR RUN THE COBOT Through DOCKER' Pictures voice_cobot
cobot_ros2_ws gesture_cobot Public yolov8n.pt
cobot_sdk_control glove_syncro5 snap
Desktop google-chrome-stable_current_amd64.deb syncro_real_control
cobot@cobot-HP-ProDesk-2-Tower-G1a-Desktop-AI-PC:~$
```

System Architecture

High-level view of complete working flow



**SAFETY
LAYER**



Open Palm
= **STOP**



No UDP Data
for 500 ms = **STOP**



Low-speed jogging
during testing



Gesture Controlled Cobot | Final Year BCA Project

ADARSH SARTHAK • Roll A 17

Data Flow Diagram (DFD)

A Data Flow Diagram (DFD) is a graphical representation of how information flows through a system. It illustrates the movement of data between different components, processes, data stores, and external entities. In this project, the DFD explains how hand gestures captured by the smart glove are converted into robot movements through wireless communication and software processing.

The Smart Glove for Intelligent Device Control consists of multiple hardware and software modules that work together to achieve real-time control of the Syncro5 Collaborative Robot. The overall system begins with gesture detection and ends with robot motion execution.

The major components involved in the data flow are:

- User
- Smart Glove
- ESP32 Transmitter
- ESP32 Receiver
- Host Computer
- Python Communication Program
- SSH Communication
- Docker Environment
- C++ Robot Controller
- Syncro5 Collaborative Robot

The following sections describe the different levels of the Data Flow Diagram.

22.1 Level 0 Data Flow Diagram (Context Diagram)

The Level 0 DFD represents the complete project as a single process. The user performs hand gestures using the smart glove. The glove captures these gestures and sends commands to the robot control system. The processed commands are finally executed by the Syncro5 Collaborative Robot.

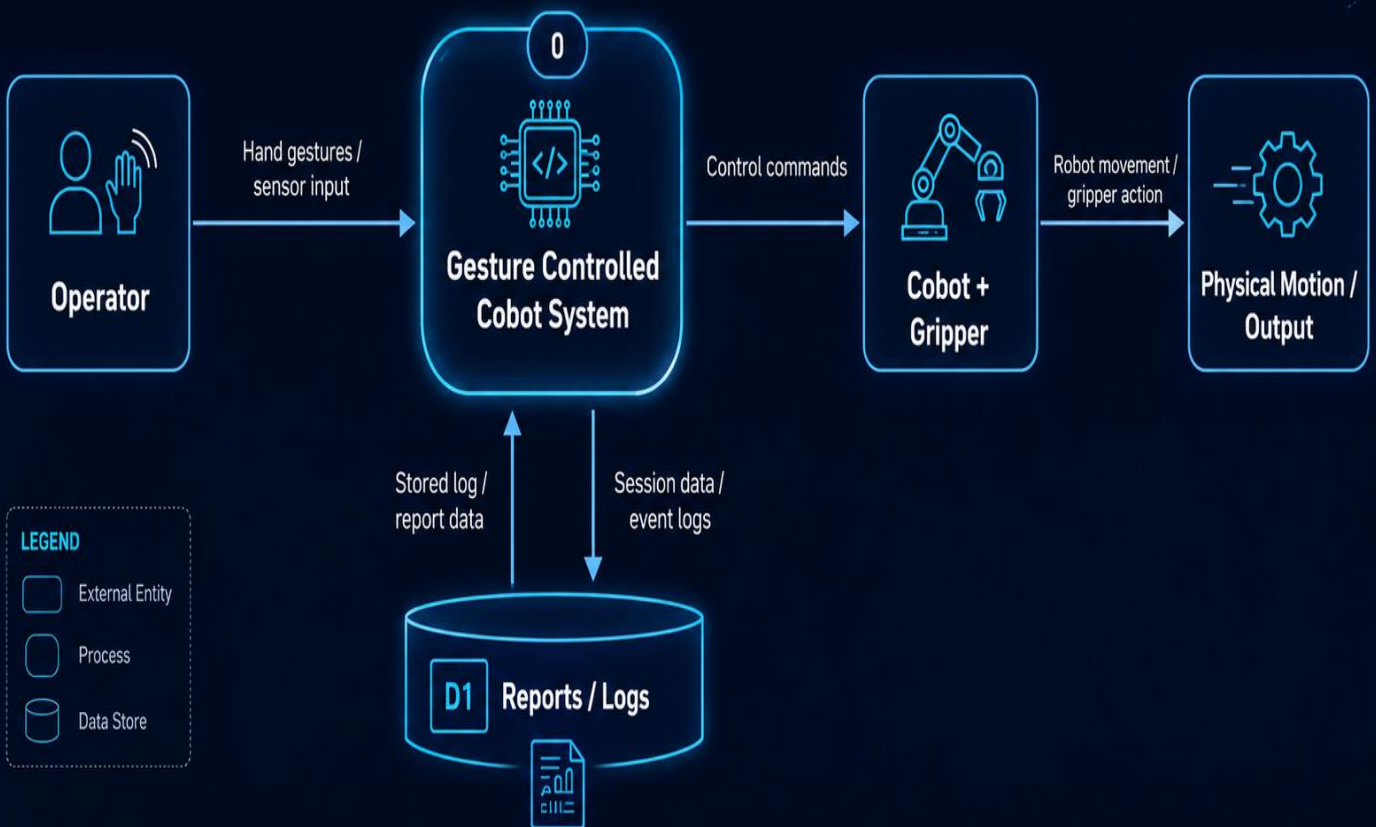
Only the overall interaction between the user and the system is represented at this level.

Figure 22.1

Level 0 Data Flow Diagram

Data Flow Diagram: Level 0

Context-level view of the gesture-controlled cobot system



Explanation

1. The operation of the Level 0 DFD can be summarized as follows:
2. The user wears the smart glove.
3. Flex sensors detect finger movements.
4. The glove control system processes the gestures.
5. Robot commands are generated.
6. The Syncro5 Cobot executes the required motion.
7. This diagram provides a high-level overview without showing internal processes.

Level 1 Data Flow Diagram

The Level 1 DFD expands the complete system into its major processing stages. It shows how gesture information

moves from the glove to the robot controller.

The major processes are:

Process 1: Gesture Detection

Flex sensors continuously measure finger bending and generate analog signals.

↓

Process 2: ESP32 Transmitter

The ESP32 reads sensor values, performs gesture recognition, and generates command characters.

↓

Process 3: ESP-NOW Communication

The generated command is transmitted wirelessly to the receiver ESP32.

↓

Process 4: ESP32 Receiver

The receiver obtains the command and forwards it to the Host PC through USB serial communication.

↓

Process 5: Python Communication Module

The Python application reads the serial data, updates the command.txt file, and transfers it to the Industrial PC through SSH.

↓

Process 6: Docker Environment

The command file becomes available inside the Docker container where the robot control application is running.

↓

Process 7: C++ Robot Controller

The controller continuously monitors the command file and calls the appropriate Syncro5 SDK functions.

↓

Process 8: Syncro5 Collaborative Robot

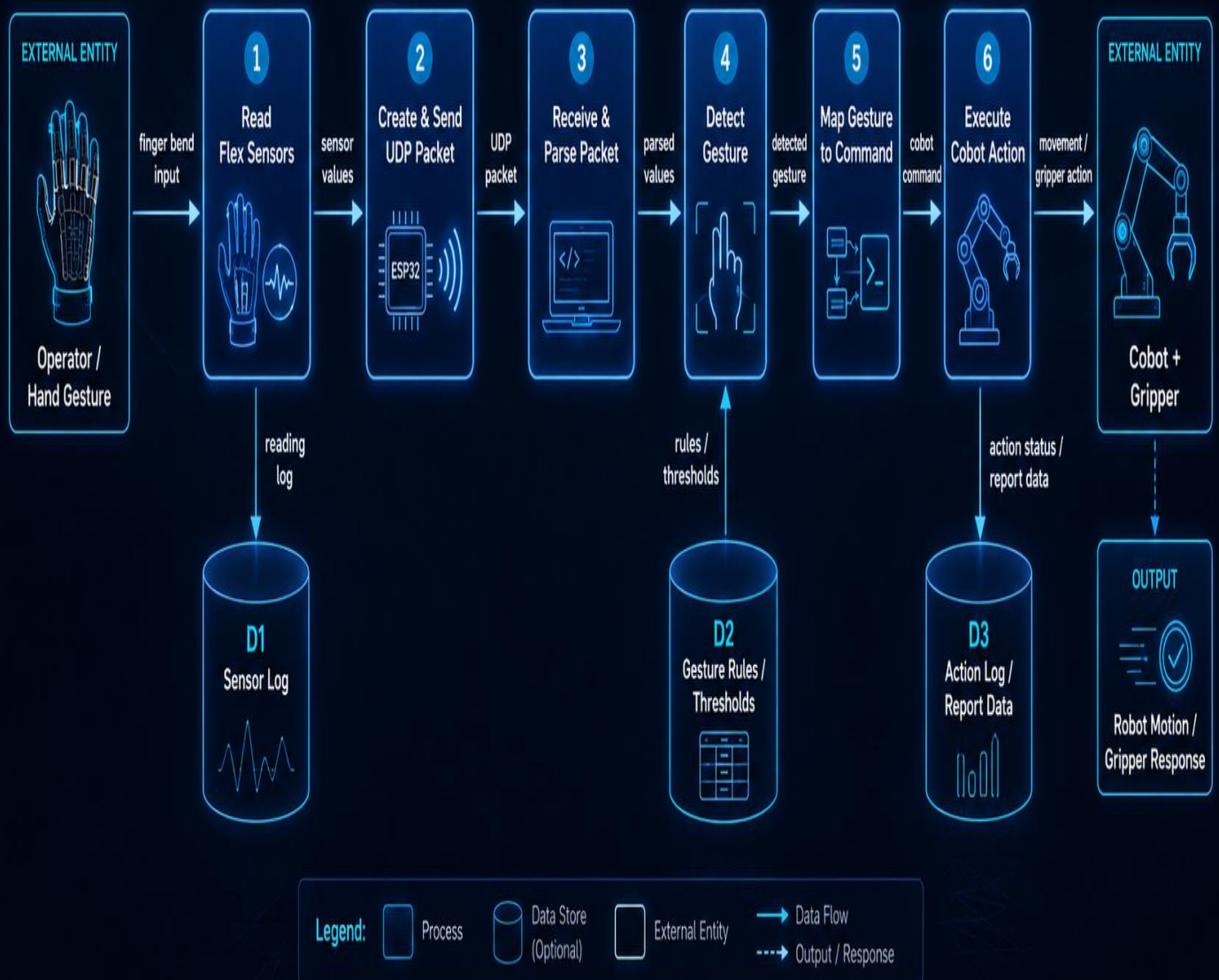
The robot executes the requested movement or gripper operation.

Figure 22.2

Level 1 Data Flow Diagram

Data Flow Diagram: Level 1

Detailed internal data flow of the gesture-controlled cobot system



Data Stores Used

The project uses one primary data store during communication.

Data Store	Description
command.txt	Stores the latest robot command generated by the Python communication program before it is read by the C++ robot controller.

External Entities

The external entities involved in the system are listed below.

Entity	Function
User	Performs hand gestures using the smart glove.
Syncro5 Cobot	Executes the received movement and gripper commands.

Data Flow Description

The complete data flow of the project is summarized below.

Source	Destination	Data Transferred
User	Smart Glove	Hand gestures
Smart Glove	ESP32 Transmitter	Flex sensor values
ESP32 Transmitter	ESP32 Receiver	Command character through ESP-NOW
ESP32 Receiver	Host PC	Serial data through USB
Host PC	command.txt	Robot command
Host PC	Industrial PC	Command file through SSH
Docker	C++ Robot Controller	Command file
C++ Robot Controller	Syncro5 Cobot	Robot motion commands

Table 22.1: Data Flow Description

Working of the Data Flow Diagram

The complete operation of the Smart Glove system begins when the user performs a hand gesture. The flex sensors mounted on the glove detect the bending of each finger and generate analog signals. These signals are processed by the ESP32 transmitter, where gesture recognition is performed.

The recognized gesture is converted into a predefined command and transmitted wirelessly using the ESP-NOW protocol. The receiver ESP32 receives the command and forwards it to the Host PC through USB serial communication.

The Python communication program running on the Host PC continuously monitors the serial port, updates the command.txt file, and securely transfers it to the Industrial PC using SSH. Inside the Docker container, the C++ Robot Controller reads the command file and invokes the appropriate Syncro5 SDK functions to execute the required robot movement or gripper action.

This continuous flow of information enables real-time control of the collaborative robot using natural hand gestures.

Flowcharts

A flowchart is a graphical representation of the sequence of operations performed by a system. It helps in understanding the logic, decision-making process, and movement of data from one stage to another.

In the Smart Glove for Intelligent Device Control, separate flowcharts are used to explain the working of the smart glove, ESP32 transmitter, ESP32 receiver, Python communication program, C++ robot controller, and the complete integrated system.

The use of flowcharts makes the system easier to understand, test, debug, and maintain.

Overall System Flowchart

The overall system flowchart represents the complete working sequence from the user's hand gesture to the final movement of the Syncro5 Collaborative Robot.

The process begins when the user wears the smart glove and performs a predefined gesture. The flex sensors detect finger bending and send analog values to the transmitter ESP32. The ESP32 identifies the gesture and sends the corresponding command to the receiver using ESP-NOW.

The receiver forwards the command to the Host PC through USB serial communication. The Python program processes the command and transfers it to the Industrial PC. The C++ robot controller reads the command and moves the robot or operates the gripper.

Flowchart Sequence

```
Start
↓
Wear Smart Glove
↓
Read Flex Sensor Values
↓
Recognize Hand Gesture
↓
Generate Command
↓
Transmit Command through ESP-NOW
↓
Receive Command on Receiver ESP32
↓
Send Command to Host PC
↓
Python Reads Serial Data
↓
Update command.txt
↓
Transfer File through SSH
```

```

↓
Docker Reads Command
↓
C++ Robot Controller Processes Command
↓
Robot Performs Required Action
↓
Repeat

```

Figure 23.1: Overall System Flowchart
(Insert complete system flowchart here.)

ESP32 Transmitter Flowchart

The ESP32 transmitter flowchart explains how the glove detects finger movement and converts it into a command. At startup, the transmitter initializes the serial communication, Wi-Fi station mode, and ESP-NOW protocol. It then performs sensor calibration while the user keeps the hand flat.

During operation, the ESP32 continuously reads all five flex sensors. The readings are filtered and compared with the stored baseline values. Based on the state of each finger, the program identifies a valid gesture and generates the corresponding command.

Flowchart Sequence

```

Start
↓
Initialize ESP32
↓
Initialize ESP-NOW
↓
Add Receiver MAC Address
↓
Calibrate Flex Sensors
↓
Read Five Sensor Values
↓
Apply EMA Filtering
↓
Compare with Baseline
↓
Determine Bent/Straight Fingers
↓
Is Gesture Valid?
├── No ──> Set Command = x
│           ↓
│           Transmit Command through ESP-NOW
│           ↓
│           Repeat Sensor Reading
└── Yes ──> Generate Command
              ↓
              Transmit Command through ESP-NOW
              ↓
              Repeat Sensor Reading

```

Main Decisions

- Whether ESP-NOW initialization is successful.
- Whether the receiver peer is added successfully.

- Whether the detected finger pattern matches a valid gesture.
- Whether the gesture is stable for the required hold time.
- Whether the open-palm stop gesture has been detected.
- Role in the Project
- Reads flex sensor values.
- Filters sensor noise.
- Recognizes hand gestures.
- Generates robot commands.
- Sends commands wirelessly.

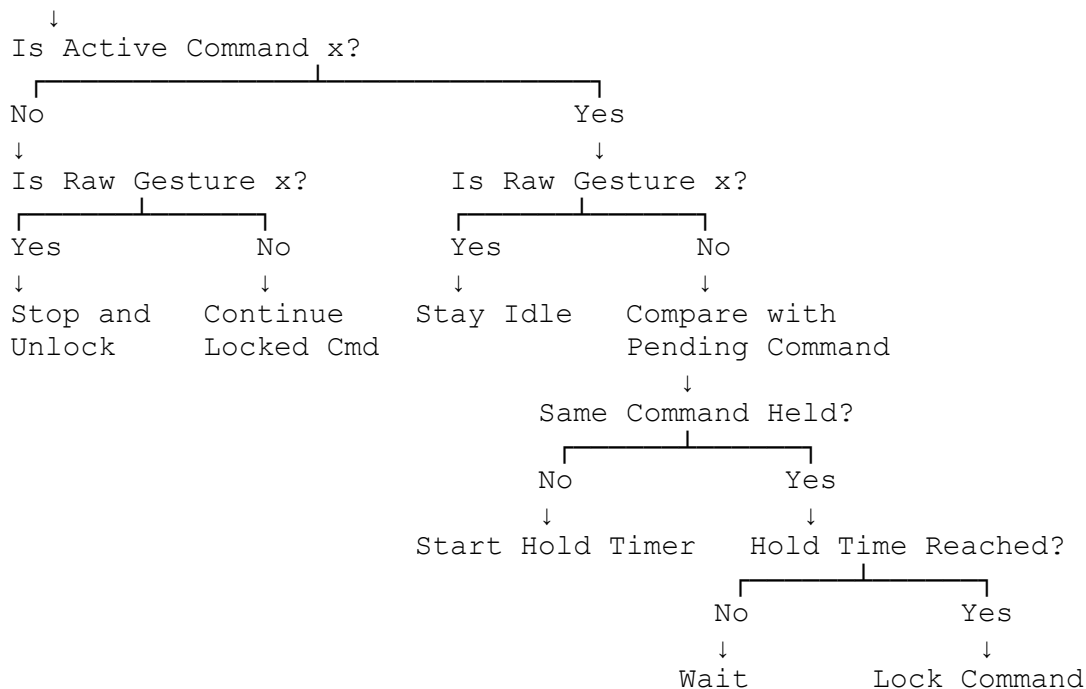
Figure 23.2: ESP32 Transmitter Flowchart
(Insert ESP32 transmitter flowchart here.)

Gesture Locking and Validation Flowchart

The gesture locking mechanism prevents unwanted command changes caused by sensor noise or accidental finger movement. A gesture must remain stable for a fixed time before it becomes an active command. Once a movement command is locked, the transmitter continues sending the same command until the user opens the hand. The open-palm gesture sends the stop command and unlocks the system.

Flowchart Sequence

Read Raw Gesture



Advantages

- Prevents false command switching.
- Reduces cross-talk between flex sensors.
- Improves robot safety.
- Ensures stable movement.

- Stops immediately when the hand is opened.

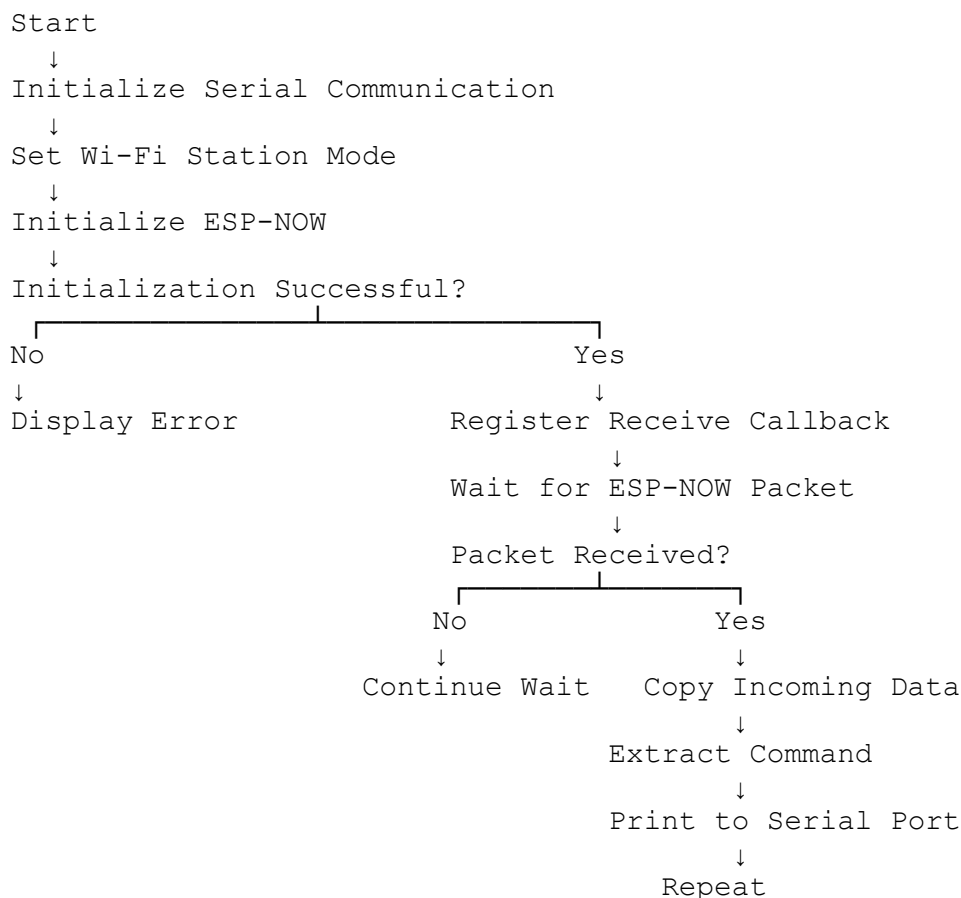
Figure 23.3: Gesture Validation and Locking Flowchart
(Insert gesture locking flowchart here.)

ESP32 Receiver Flowchart

The receiver ESP32 acts as the communication interface between the wireless transmitter and the Host PC. The receiver initializes ESP-NOW and registers a callback function. Whenever a command packet is received, the callback extracts the command and prints it through the serial port.

The receiver does not modify the command. This keeps the communication fast and reduces processing delay.

Flowchart Sequence



Role in the Project

- Receives commands from the glove.
- Extracts the command character.
- Sends the command to the Host PC.
- Maintains continuous wireless reception.

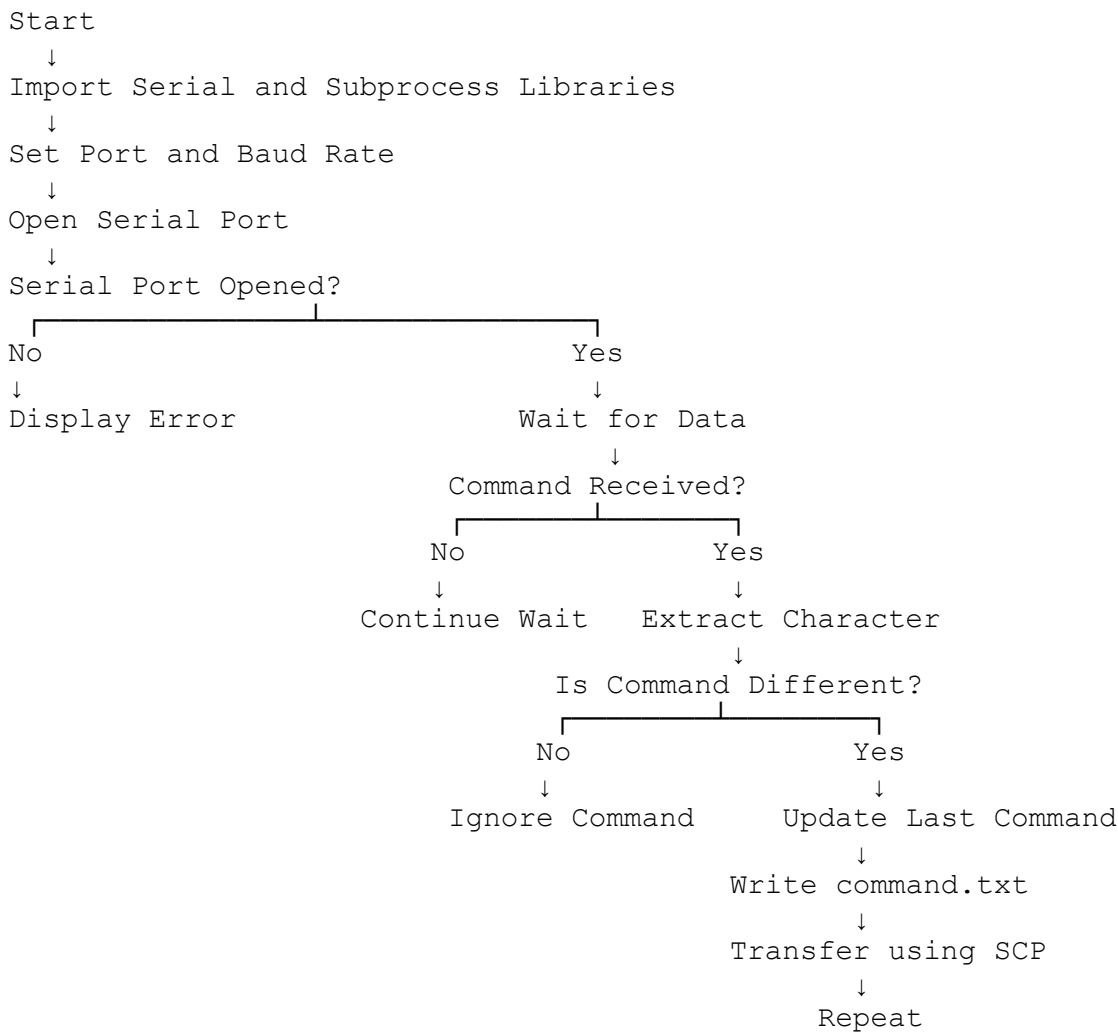
Figure 23.4: ESP32 Receiver Flowchart
(Insert ESP32 receiver flowchart here.)

Python Communication Flowchart

The Python communication program connects the ESP32 receiver to the Industrial PC. It opens the serial port and waits for incoming data.

Whenever a command is received, the program extracts the first character and compares it with the previous command. If the command has changed, it writes the new command into `command.txt` and transfers the file to the Industrial PC through SCP.

The command comparison prevents repeated file transfers when the same gesture is continuously received.
Flowchart Sequence



Role in the Project

- Reads serial commands.
- Filters repeated commands.
- Creates and updates the command file.
- Transfers commands to the Industrial PC.
- Connects embedded hardware with robot software.

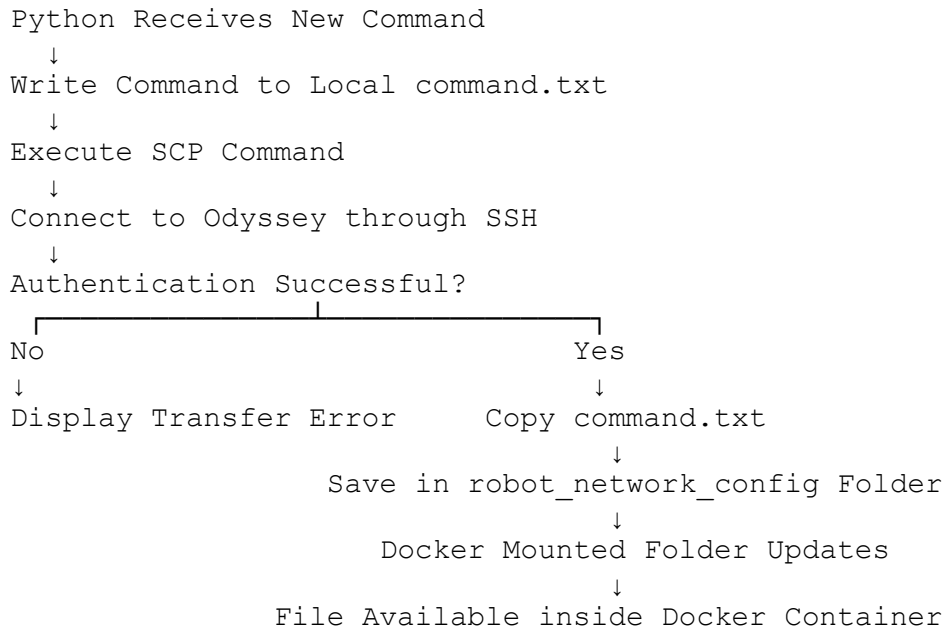
Figure 23.5: Python Communication Flowchart

Host PC to Docker Flowchart

This flowchart explains how the command generated on the Host PC becomes available inside the Docker container.

The Python application transfers the command file to the `robot_network_config` directory on the Odyssey Industrial PC. This folder is mounted inside the Docker container. Therefore, the same command file becomes accessible inside Docker at `/robot_network_config/command.txt`.

Flowchart Sequence



Important Paths

Host PC:

`~/glove_syncro5/pc_reader/command.txt`

Odyssey Host:

`/home/cobot/robot_network_config/command.txt`

Docker Container:

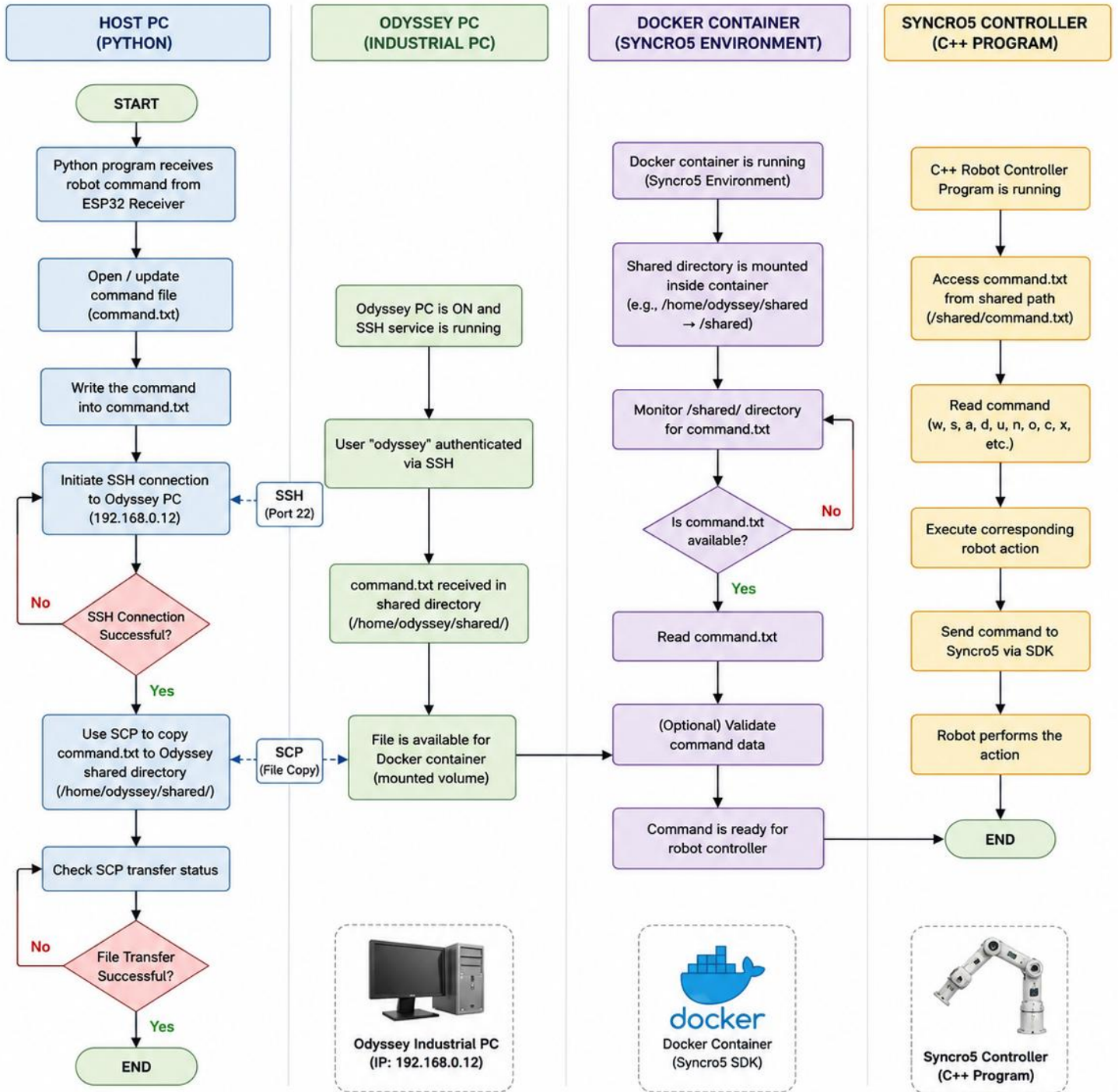
`/robot_network_config/command.txt`

Role in the Project

- Transfers commands securely.
- Provides a bridge between two computers.
- Makes the command accessible inside Docker.
- Avoids direct serial dependency inside the container.

Figure 23.6: Host PC to Docker Flowchart

SSH AND DOCKER FILE-SHARING FLOWCHART



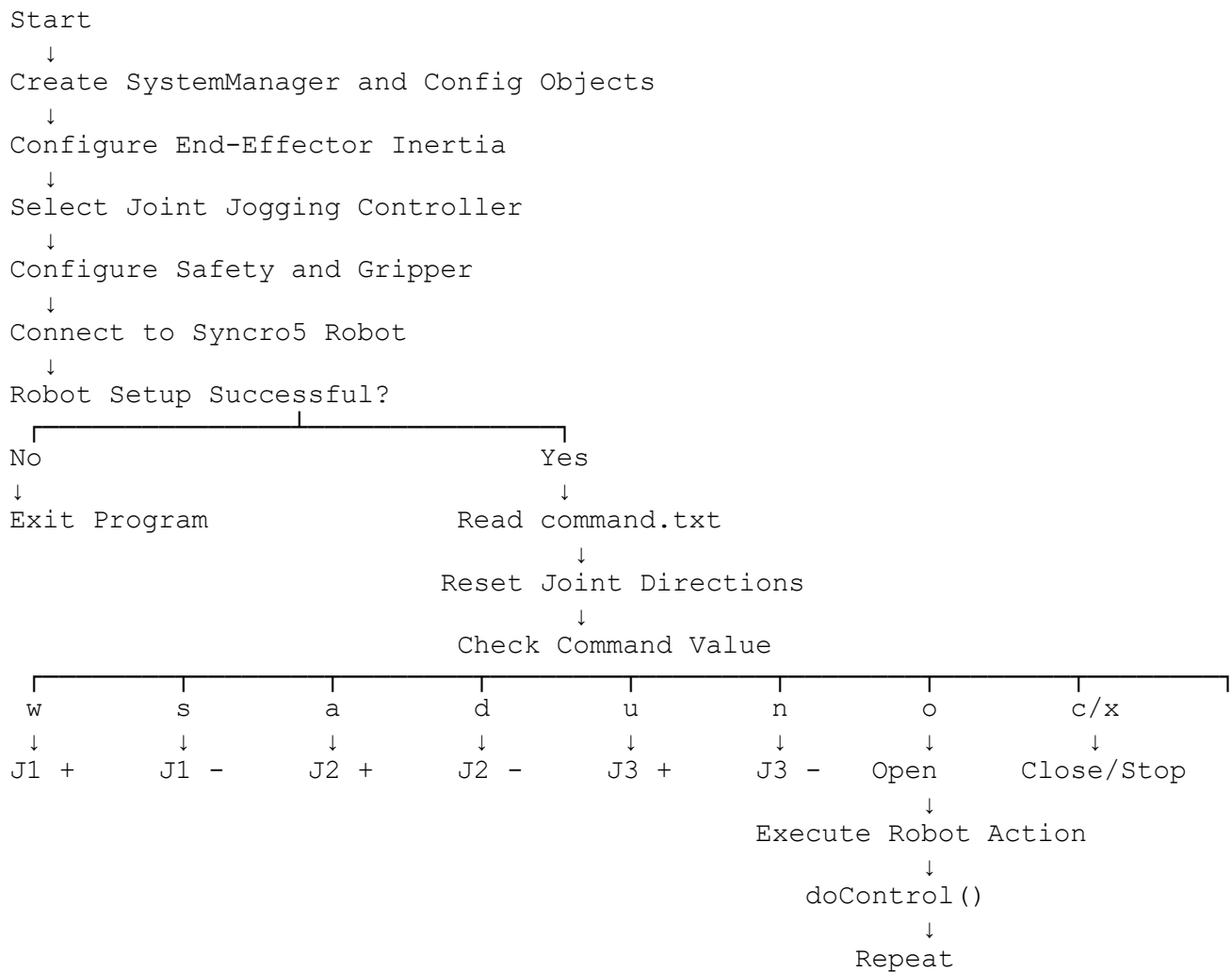
C++ Robot Controller Flowchart

The C++ robot controller is executed inside the Docker container. It initializes the robot, configures the joint jogging controller, and continuously reads the command file.

Before processing each command, all joint jogging directions are reset to zero. The received command is then matched with the required robot action.

Commands **w**, **s**, **a**, **d**, **u**, and **n** control robot movement. Commands **o** and **c** control the gripper, while **x** stops the robot.

Flowchart Sequence



Command Decisions

Command	Controller Action
w	Joint 1 positive movement
s	Joint 1 negative movement

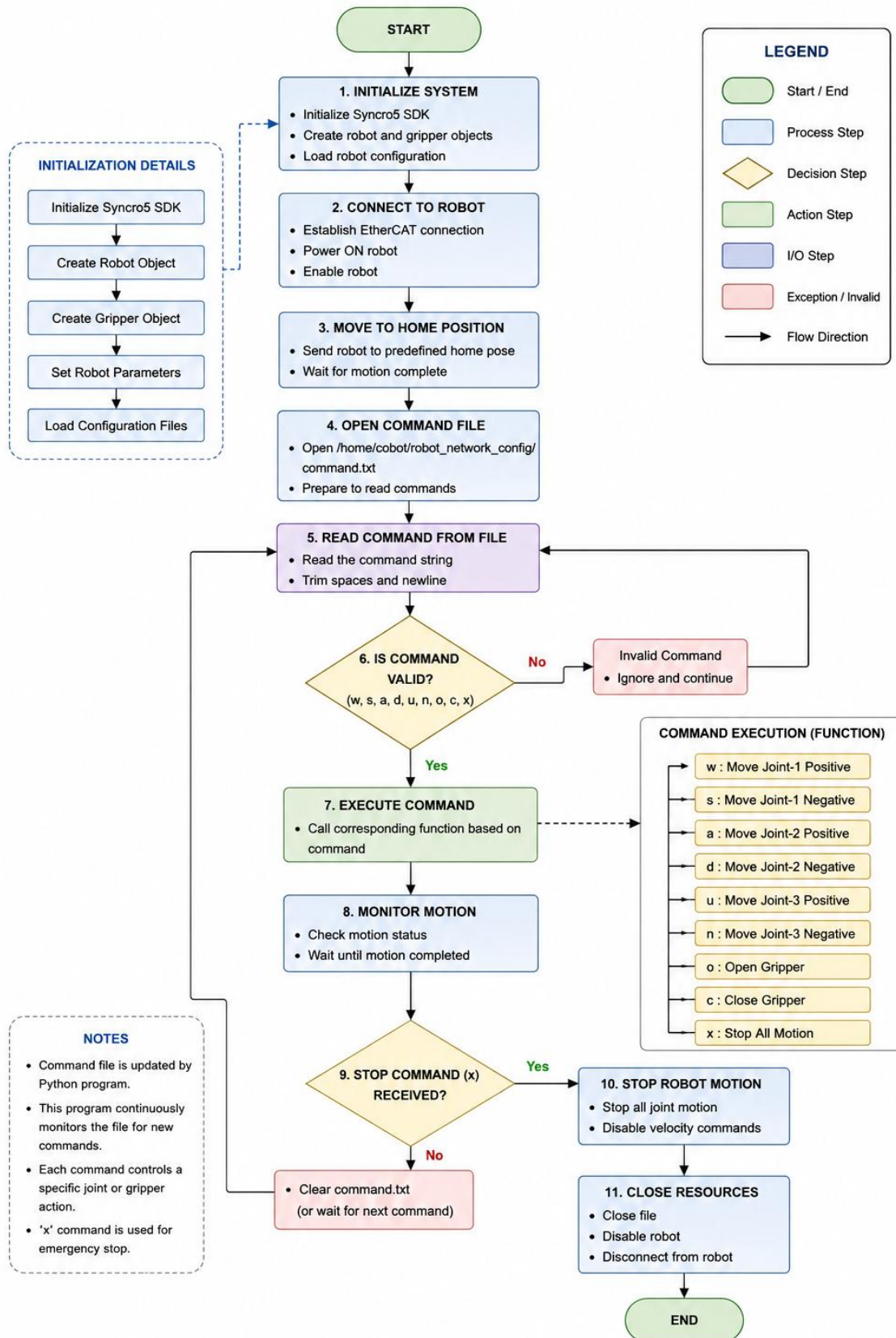
a	Joint 2 positive movement
d	Joint 2 negative movement
u	Joint 3 positive movement
n	Joint 3 negative movement
o	Open gripper
c	Close gripper
x	Stop all movement

Role in the Project

- Connects to the Syncro5 Robot SDK.
- Reads the latest gesture command.
- Executes joint movement.
- Controls the gripper.
- Stops robot motion when required.

Figure 23.7: C++ Robot Controller Flowchart

C++ ROBOT CONTROL FLOWCHART

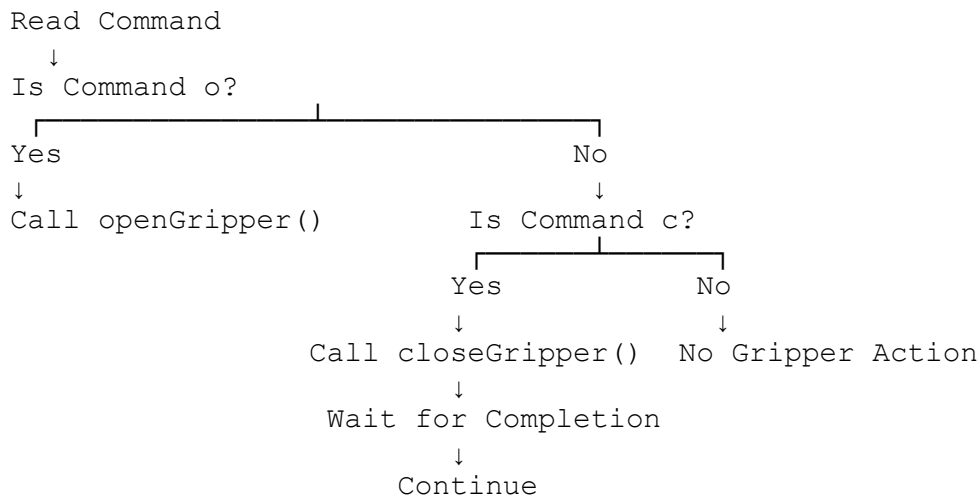


Gripper Control Flowchart

The gripper is controlled using two gesture commands. The `o` command opens the gripper, while the `c` command closes it.

The robot controller checks the gripper command and calls the corresponding SDK function. A short delay is added after each operation to allow the gripper to complete the movement.

Flowchart Sequence



SDK Functions Used

- `sys.openGripper();`
- `sys.closeGripper(130);`

Role in the Project

- Allows object gripping.
- Supports pick-and-place operations.
- Provides gesture-based end-effector control.

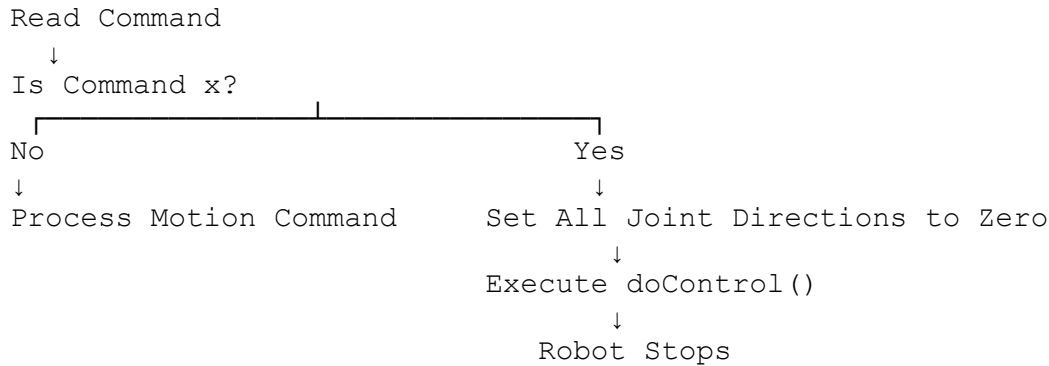
Figure 23.8: Gripper Control Flowchart

Stop and Safety Flowchart

Safety is important because the Syncro5 is a real industrial collaborative robot. The open-palm gesture generates the `x` command, which stops all commanded joint movement.

The C++ controller sets every joint jogging direction to zero before each control cycle. This ensures that the robot stops when the stop command is received or when no valid movement command is available.

Flowchart Sequence



Safety Measures

1. Open palm used as stop gesture.
2. Joint commands reset before every loop.
3. Invalid commands do not create movement.
4. Robot controller operates with configured safety type.
5. User can terminate the program using CTRL + C.
6. Gripper thermal protection warning is monitored.

Module Description

The Smart Glove for Intelligent Device Control is developed using a modular architecture, where the complete system is divided into several independent modules. Each module performs a specific task and communicates with the next module to achieve real-time control of the collaborative robot.

The modular design simplifies system development, testing, debugging, maintenance, and future expansion. Since every module works independently, modifications can be made to one module without affecting the overall system. The major modules of the project are described below.

Module 1: Smart Glove Module

The Smart Glove Module is the primary input device of the system. It is designed using a leather glove fitted with five flex sensors, an ESP32 transmitter, a rechargeable battery pack, and the required electronic circuitry. Whenever the user bends or straightens a finger, the corresponding flex sensor changes its resistance. These changes are converted into analog voltage signals and supplied to the ESP32 transmitter for processing.

Functions

- Detect finger movements
- Capture hand gestures
- Generate analog sensor data
- Provide user interaction
- Input
- Finger bending and hand gestures

- Output
- Analog values from five flex sensors
- Components Used
- Leather glove
- Five flex sensors
- ESP32 transmitter
- 10 k Ω resistors
- Lithium-ion battery
- Charging module

Figure 24.1: Smart Glove Module



Module 2: Gesture Recognition Module

The Gesture Recognition Module processes the analog values received from the flex sensors. It applies signal filtering, compares the sensor readings with calibrated baseline values, and identifies the performed gesture. After successful recognition, the detected gesture is converted into a predefined command character.

Functions

- Read sensor values

- Filter sensor noise
- Perform calibration
- Recognize hand gestures
- Generate command characters
- Input
- Analog flex sensor readings
- Output
- Robot command

Commands Generated

- $x \rightarrow \text{Stop}$
- $w \rightarrow +X$
- $s \rightarrow -X$
- $a \rightarrow +Y$
- $d \rightarrow -Y$
- $u \rightarrow +Z$
- $n \rightarrow -Z$
- $\rightarrow \text{Open Gripper}$
- $c \rightarrow \text{Close Gripper}$

Figure 24.2: Gesture Recognition Process

Module 3: Wireless Communication Module

The Wireless Communication Module transfers commands from the transmitter ESP₃₂ to the receiver ESP₃₂ using the ESP-NOW protocol.

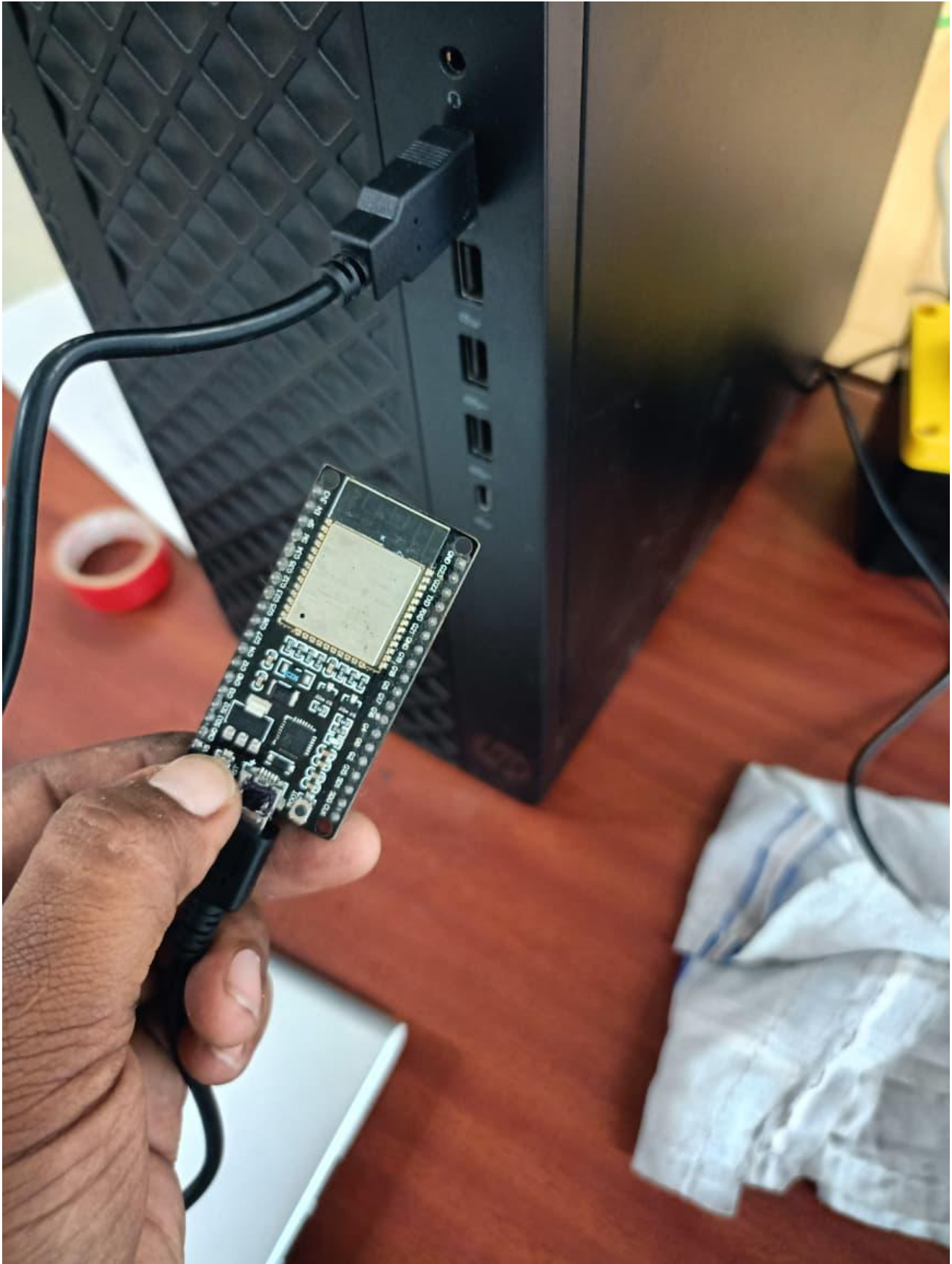
ESP-NOW provides low-latency communication without requiring a Wi-Fi router, making it suitable for real-time robotic control.

Functions

- Wireless command transmission
- Packet reception
- Reliable communication
- Input
- Command character
- Output
- Wireless data packet
- Technologies Used
- ESP-NOW
- ESP₃₂ transmitter

- ESP₃₂ receiver

Figure 24.3: ESP-NOW Communication



Module 4: Host PC Communication Module

The Host PC receives command characters from the receiver ESP32 through USB serial communication. A Python application continuously monitors the serial port and stores the latest command in a text file. Whenever the command changes, the updated file is transferred to the Industrial PC using SSH.

Functions

- Read serial data
- Update command file
- Transfer command through SSH
- Synchronize communication
- Input
- Serial command
- Output
- Updated `command.txt`
- Software Used
- Python
- PySerial
- SCP
- SSH

Figure 24.4: Host PC Communication
(Insert Host PC communication diagram.)

Module 5: Robot Control Module

The Robot Control Module runs inside the Docker container on the Industrial PC. It continuously monitors the command file and executes the corresponding robot movement using the Syncro5 SDK. Depending on the received command, the controller performs joint jogging or gripper operations.

Functions

- Read command file
- Initialize robot
- Execute robot motion
- Operate gripper
- Input
- Command file
- Output
- Robot movement
- SDK Functions Used
- `setupRobot()`
- `doControl()`

- `openGripper()`
- `closeGripper()`

Figure 24.5: Robot Control Module
(Insert robot control workflow.)

Module 6: Collaborative Robot Module

The Collaborative Robot Module represents the final execution stage of the system. After receiving commands from the controller, the Syncro5 Collaborative Robot performs the required movement.

The robot can move along different joint directions and operate the Feetech gripper to perform object manipulation tasks.

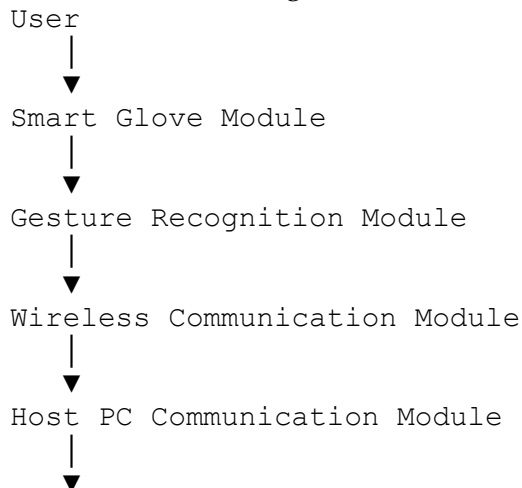
Functions

- Execute robot motion
- Perform joint jogging
- Open and close gripper
- Execute pick-and-place operations
- Input
- Robot motion commands
- Output
- Physical robot movement
- Components
- Syncro5 Collaborative Robot
- Feetech Electric Gripper

Figure 24.6: Syncro5 Collaborative Robot
(Insert photograph of the robot.)

Module Interaction

The interaction among all modules is illustrated below.



Robot Control Module



Collaborative Robot Module

Figure 24.7: Module Interaction Diagram

(Insert module interaction block diagram.)

Implementation

The implementation phase of the Smart Glove for Intelligent Device Control involved integrating hardware components, communication protocols, software applications, and the collaborative robot into a single working system. The project was implemented in multiple stages, beginning with the development of the smart glove and ending with successful real-time control of the Syncro5 Collaborative Robot.

A modular implementation approach was adopted to simplify testing, debugging, and system integration. Each module was individually verified before being connected with the complete system.

Smart Glove Development

The first stage of implementation involved designing the wearable smart glove. Five 2.2-inch flex sensors were mounted on a leather glove, with one sensor attached to each finger. The sensors were fixed securely using white paper tape and hot glue to prevent movement during operation.

Each flex sensor was connected to the ESP32 transmitter through a voltage divider circuit using 10 kΩ resistors. The ESP32 board, rechargeable lithium-ion batteries, and charging module were mounted on the glove to create a portable and self-powered device.

Activities Performed

- Mounted flex sensors on the glove.
- Connected voltage divider circuits.
- Installed ESP32 transmitter.
- Connected rechargeable battery pack.
- Tested sensor outputs.

Figure 25.1: Developed Smart Glove

(Insert photograph of the completed glove.)

ESP32 Programming

After assembling the hardware, both ESP32 boards were programmed using the Arduino IDE.

The transmitter ESP32 continuously reads the flex sensor values, performs gesture recognition, and transmits commands using ESP-NOW. The receiver ESP32 listens for incoming commands and forwards them to the Host PC through USB serial communication.

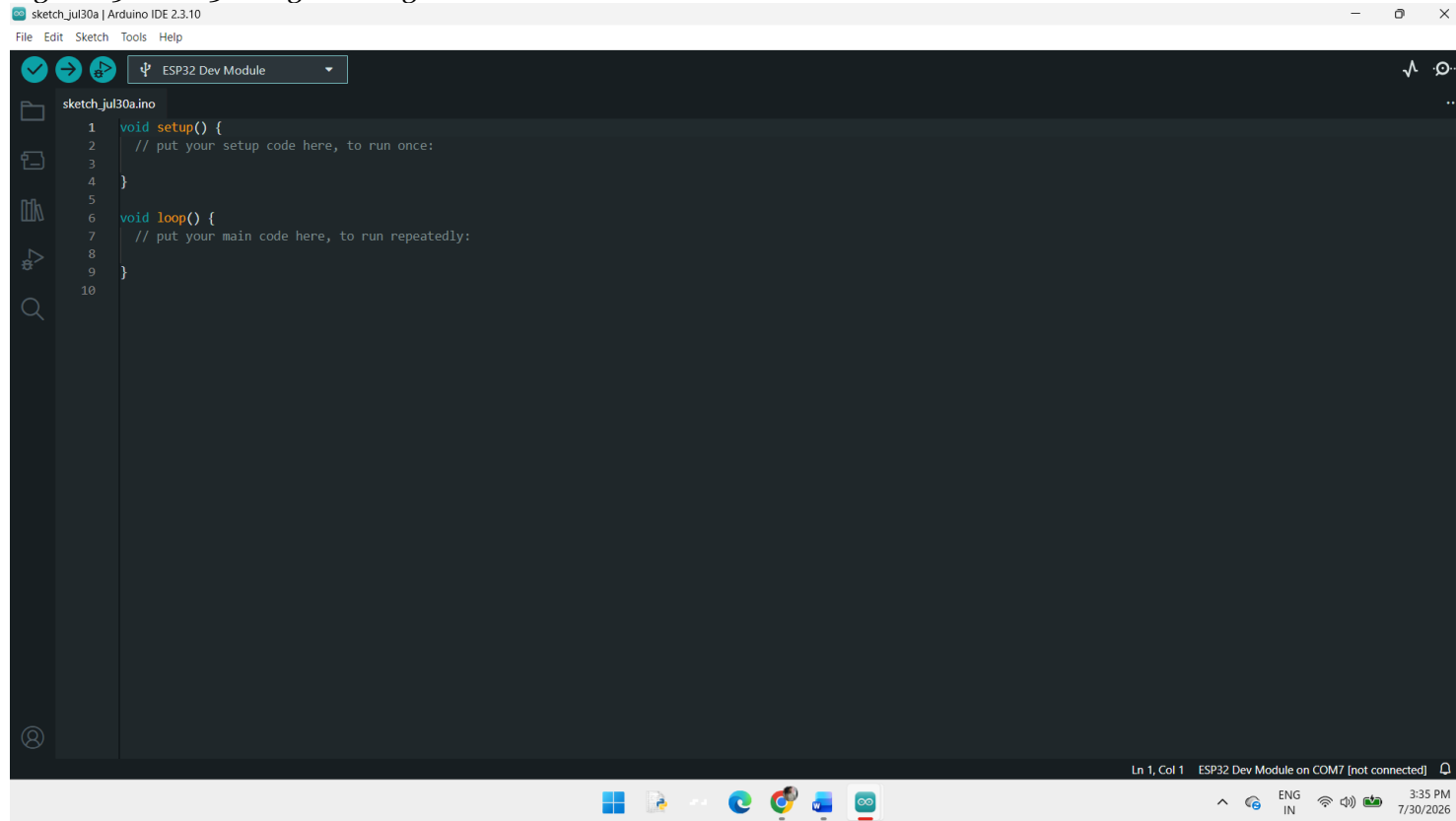
Each gesture command was tested individually to verify correct wireless communication.

Activities Performed

- Programmed transmitter ESP32.
- Programmed receiver ESP32.

- Configured ESP-NOW communication.
- Added receiver MAC address.
- Verified wireless data transmission.

Figure 25.2: ESP₃₂ Programming



Communication Between Host PC and Industrial PC

The receiver ESP₃₂ was connected to the Host PC using a USB cable. A Python program continuously monitored the serial port and received gesture commands from the receiver.

Whenever a new command was detected, the Python application updated the `command.txt` file and transferred it to the Industrial PC using Secure Shell (SSH) and Secure Copy Protocol (SCP).

Before implementing this communication, SSH authentication and network connectivity between the Host PC and the Industrial PC were successfully configured.

Activities Performed

- Connected receiver ESP₃₂.
- Configured serial communication.
- Established SSH connection.
- Enabled automatic file transfer.
- Verified successful command transmission.

Figure 25.3: Host PC Communication

```
cobot@cobot-HP-ProDesk-2-Tower-G1a-Desktop-AI-PC: ~/glove_syncro5
cobot@cobot-HP-ProDesk-2-Tower-G1a-Desktop-AI-PC: ~/glove_syncro5 122x33
cobot@cobot-HP-ProDesk-2-Tower-G1a-Desktop-AI-PC:~$ ls
'ADDVERB TEAM MAIL ID'  Documents      install        Templates
Arduino                 Downloads      log            'Untitled 1.odt'
build                   dual_cam_pick  Music          Videos
cap_%d                  'FOR RUN THE COBOT Through DOCKER'  Pictures       voice_cobot
cobot_ros2_ws           gesture_cobot  Public         yolov8n.pt
cobot_sdk_control       glove_syncro5  snap
Desktop                 google-chrome-stable_current_amd64.deb syncro_real_control
cobot@cobot-HP-ProDesk-2-Tower-G1a-Desktop-AI-PC:~$ cd glove_syncro5/
cobot@cobot-HP-ProDesk-2-Tower-G1a-Desktop-AI-PC:~/glove_syncro5$ ls
esp32_receiver  pc_reader
cobot@cobot-HP-ProDesk-2-Tower-G1a-Desktop-AI-PC:~/glove_syncro5$
```

Docker-Based Robot Control

The Syncro5 Robot SDK operates inside a Docker container running on the Industrial PC. A C++ robot controller application was developed and compiled using CMake and GCC.

The controller continuously monitors the transferred `command.txt` file. Whenever a new command becomes available, the application executes the corresponding robot movement using the Syncro5 SDK APIs.

Both joint jogging and gripper operations were successfully implemented and tested.

Activities Performed

- Created Docker environment.
- Configured Syncro5 SDK.
- Compiled C++ application.
- Connected robot controller.
- Executed robot commands.

Figure 25.4: Docker Environment
(Insert Docker terminal screenshot.)

Robot Motion Implementation

After successful communication between all modules, the gesture commands were mapped to robot movements. Each recognized gesture generated a predefined command that controlled one of the robot joints or the gripper. The robot continuously monitored incoming commands and responded in real time.

The implemented robot operations include:

- Positive X movement
- Negative X movement
- Positive Y movement

- Negative Y movement
- Positive Z movement
- Negative Z movement
- Gripper opening
- Gripper closing
- Robot stop

Each operation was verified individually before testing the complete system.

Figure 25.5: Robot Motion Testing



System Integration

Once all individual modules were tested successfully, they were integrated into a single system.

- The complete workflow operates as follows:
- The user performs a hand gesture.
- Flex sensors detect finger movement.
- ESP32 transmitter recognizes the gesture.
- Command is transmitted using ESP-NOW.
- Receiver ESP32 forwards the command to the Host PC.
- Python application updates the command file.
- SSH transfers the command to the Industrial PC.
- Docker-based C++ controller reads the command.
- Syncro5 SDK executes the corresponding robot movement.

This integrated implementation enables real-time control of the collaborative robot through natural hand gestures.

Figure 25.6: Complete System Integration

(Insert complete system architecture diagram.)

Implementation Challenges

During the implementation process, several technical challenges were encountered. These issues were identified, analyzed, and resolved through systematic testing.

Some of the major challenges included:

- Flex sensor noise and unstable readings.
- Incorrect gesture detection.
- ESP-NOW communication issues.
- SSH permission and file transfer errors.
- Docker file access problems.
- C++ compilation and SDK integration errors.
- Robot communication and controller configuration.
- Gripper overheating protection during continuous operation.

Each issue was resolved through software optimization, proper hardware configuration, and extensive testing.

Final Implementation Result

After completing the implementation and testing phases, the developed system successfully demonstrated real-time gesture-based control of the Syncro5 Collaborative Robot.

The system achieved:

1. Accurate gesture recognition.
2. Reliable wireless communication.
3. Stable SSH-based command transfer.
4. Successful Docker integration.
5. Real-time robot motion control.
6. Gesture-based gripper operation.
7. Smooth overall system performance.

The final implementation validates the feasibility of using a wearable smart glove as a natural and intuitive interface for controlling industrial collaborative robots.

Testing and Validation

Testing and validation are essential phases of any engineering project to ensure that the developed system performs accurately, reliably, and safely under different operating conditions. The Smart Glove for Intelligent Device Control was tested in multiple stages, beginning with individual hardware components and ending with complete system integration.

Each module was tested independently before integrating it with the complete system. This modular testing approach helped identify errors at an early stage and improved the overall reliability of the system. The major testing stages carried out during the project are described below.

Flex Sensor Testing

The first stage involved testing the flex sensors individually. Each sensor was connected to the ESP32 transmitter and monitored using the Arduino Serial Monitor.

The analog values were observed while bending and straightening each finger. The obtained readings were used to determine suitable threshold values for gesture recognition.

Objectives

- Verify sensor operation.
- Measure analog output values.
- Identify stable threshold values.
- Check response to finger bending.
- Expected Result
- Each finger should produce a noticeable change in analog value when bent.
- Obtained Result

All five flex sensors responded correctly and generated stable analog readings.

Status: Passed ✓

Figure 26.1: Flex Sensor Testing

Gesture Recognition Testing

After sensor testing, gesture recognition was verified.

Different hand gestures were performed repeatedly, and the ESP32 transmitter correctly generated the predefined command characters.

Each gesture was tested multiple times to ensure consistent recognition.

Objectives

- Verify gesture detection.
- Validate command generation.
- Check recognition accuracy.

Tested Gestures

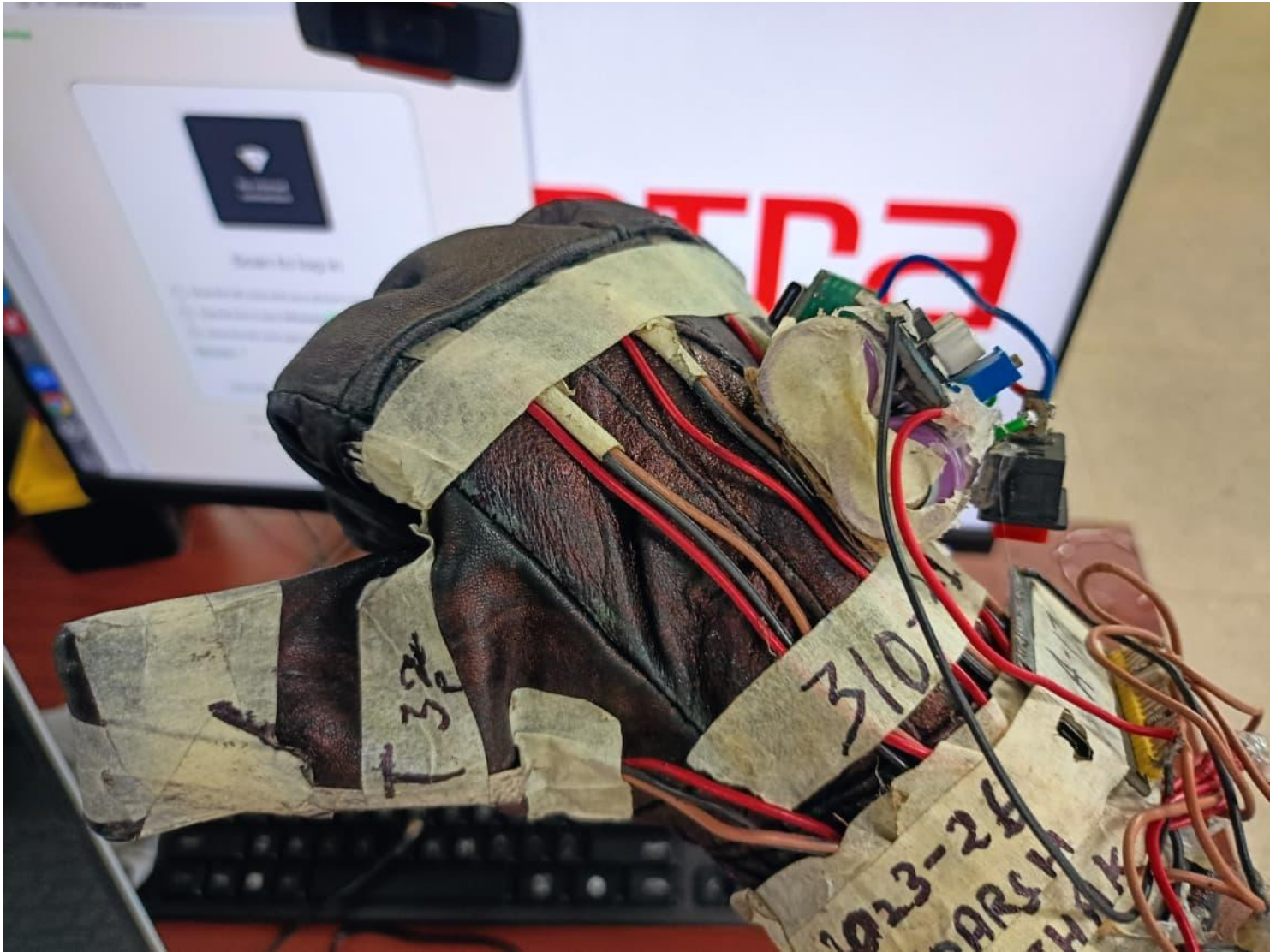
- Stop
- +X
- -X
- +Y
- -Y
- +Z
- -Z
- Open Gripper
- Close Gripper

Expected Result

- Correct command should be generated for every predefined gesture.
- Obtained Result
- The system successfully recognized all programmed gestures.
- Status: Passed ✓

Figure 26.2: Gesture Recognition Testing





ESP-NOW Communication Testing

The ESP-NOW communication link between the transmitter and receiver ESP32 boards was tested. Commands generated by the transmitter were transmitted wirelessly and verified on the receiver through the Serial Monitor.

Objectives

- Verify wireless communication.
- Check packet transmission.
- Measure communication reliability.
- Expected Result
- Receiver should receive every transmitted command without noticeable delay.
- Obtained Result
- All commands were successfully received with low communication latency.

- Status: Passed ✓

Figure 26.3: ESP-NOW Communication Testing
(Insert receiver Serial Monitor screenshot.)

Serial Communication Testing

The receiver ESP32 was connected to the Host PC through USB serial communication. The Python application continuously monitored the serial port and displayed the received command characters.

Objectives

- Verify USB serial communication.
- Check command reception.
- Detect communication errors.
- Expected Result
- Python program should receive every command from the receiver.
- Obtained Result
- The serial communication worked correctly without data loss.
- Status: Passed ✓

Figure 26.4: Serial Communication Testing
(Insert Python terminal screenshot.)

SSH File Transfer Testing

The communication between the Host PC and the Industrial PC was tested using Secure Shell (SSH). Whenever a new gesture command was detected, the Python program automatically updated the command.txt file and transferred it to the Industrial PC using SCP.

Objectives

- Verify SSH connectivity.
- Validate automatic file transfer.
- Check command synchronization.
- Expected Result
- Updated command file should appear on the Industrial PC immediately after a gesture is detected.
- Obtained Result
- Automatic file transfer was completed successfully.
- Status: Passed ✓

Figure 26.5: SSH Communication Testing
(Insert SCP terminal screenshot.)

Docker Communication Testing

The Docker container was tested to verify that it could access the transferred `command.txt` file. The C++ test program continuously monitored the file and displayed the received commands.

Objectives

- Verify shared file access.
- Validate Docker communication.
- Test command reading.
- Expected Result
- The Docker application should detect every updated command.
- Obtained Result
- The C++ application successfully read every transferred command.
- Status: Passed ✓

Figure 26.6: Docker Command Testing
(Insert Docker terminal screenshot.)

Robot Motion Testing

The C++ robot controller was executed inside the Docker container, and each gesture command was verified individually.

The corresponding robot joint moved correctly according to the generated command.

Objectives

- Verify robot movement.
- Validate command mapping.
- Test real-time response.
- Tested Motions
- Joint 1 Positive
- Joint 1 Negative
- Joint 2 Positive
- Joint 2 Negative
- Joint 3 Positive
- Joint 3 Negative
- Stop

Expected Result

- Robot should perform the correct movement corresponding to each gesture.
- Obtained Result
- All programmed robot movements executed successfully.
- Status: Passed ✓

Figure 26.7: Robot Motion Testing

Gripper Testing

The gripper control commands were tested independently.

The open and close gestures successfully operated the Feetech electric gripper.

Objectives

- Verify gripper operation.
- Check command execution.
- Validate SDK functions.

Expected Result

- The gripper should open and close according to the detected gesture.
- Obtained Result
- Both gripper operations were performed successfully.
- Status: Passed ✓

Figure 26.8: Gripper Testing
(Insert gripper operation photographs.)

Integrated System Testing

Finally, the complete system was tested by integrating all hardware and software modules. The user controlled the Syncro5 Collaborative Robot using the smart glove, while all communication stages operated simultaneously.

The overall system successfully demonstrated real-time gesture-based robot control.

Objectives

- Verify complete system integration.
- Validate end-to-end communication.
- Test overall system performance.

Expected Result

- Hand gestures should control the collaborative robot smoothly and accurately.
- Obtained Result
- The integrated system successfully performed real-time robot control with stable communication and reliable operation.
- Status: Passed ✓

Figure 26.9: Complete System Testing

Test Result Summary

Test Case	Expected Result	Obtained Result	Status
Flex Sensor Testing	Correct analog values	Successful	Passed

Gesture Recognition	Correct command generation	Successful	Passed
ESP-NOW Communication	Reliable wireless transmission	Successful	Passed
Serial Communication	Correct command reception	Successful	Passed
SSH File Transfer	Automatic file synchronization	Successful	Passed
Docker Communication	Correct file access	Successful	Passed
Robot Motion	Correct joint movement	Successful	Passed
Gripper Operation	Open and close successfully	Successful	Passed
Complete System Integration	End-to-end operation	Successful	Passed

Table 26.1: Test Result Summary

Validation

The developed system was validated under practical operating conditions using the Syncro5 Collaborative Robot. Multiple gesture combinations were executed repeatedly to evaluate system stability, communication reliability, and robot response.

The experimental results confirmed that:

- The flex sensors accurately detected finger movements.
- The ESP32 transmitter correctly recognized predefined gestures.
- ESP-NOW provided fast and reliable wireless communication.
- The Python communication module successfully synchronized commands with the Industrial PC.
- The Docker-based C++ application correctly interpreted the commands.
- The Syncro5 Collaborative Robot executed the corresponding joint movements and gripper operations with minimal delay.

The complete system operated reliably in real-time without significant communication failures during testing. The successful completion of all testing stages validates the effectiveness of the proposed Smart Glove for Intelligent Device Control and demonstrates its suitability for intuitive, gesture-based control of collaborative robots and other intelligent devices

Problems Faced and Solutions

During the development of the Smart Glove for Intelligent Device Control, several hardware, software, communication, and robot integration challenges were encountered. Since the project involved embedded systems, wireless communication, Linux, Docker, Python, C++, and an industrial collaborative robot, careful testing and debugging were required to achieve reliable system performance.

The major problems faced during the project and their corresponding solutions are described below.

Problem 1: Unstable Flex Sensor Readings

Initially, the flex sensor readings fluctuated continuously even when the fingers remained stationary. This caused incorrect gesture detection and unstable robot movement.

Cause

- Electrical noise
- Sensor sensitivity
- Small finger vibrations

Solution

- An Exponential Moving Average (EMA) filter was implemented to smooth the sensor values. Automatic calibration was also added to record the baseline value of each sensor when the hand was fully open.

Result

- Stable sensor readings
 - Improved gesture recognition accuracy
 - Reduced false detections
-

Problem 2: False Gesture Detection

During initial testing, unintended finger movements occasionally triggered incorrect commands because several finger combinations had similar sensor values.

Cause

- Finger cross-talk
- Similar analog readings
- Sudden finger movement

Solution

- A gesture validation and lock mechanism was introduced. The system accepts a command only after the gesture remains stable for a predefined hold time. Once a movement command is locked, it remains active until the user performs the stop gesture.

Result

- Reliable gesture recognition
 - Fewer accidental commands
 - Improved safety
-

Problem 3: ESP-NOW Communication Failure

The ESP32 transmitter initially failed to communicate with the receiver.

Cause

- Incorrect receiver MAC address
- ESP-NOW initialization issues
- Peer configuration errors

Solution

- The receiver MAC address was verified using a dedicated ESP32 program. The correct MAC address was added to the transmitter program, and ESP-NOW peer configuration was reinitialized.

Result

- Successful wireless communication
 - Reliable command transmission
-

Problem 4: Serial Communication Error

The Python application was unable to read data from the ESP32 receiver.

Cause

- Incorrect serial port
- Receiver not detected
- USB connection problem

Solution

- The correct serial port was identified using Linux device commands, and the receiver was reconnected. The Python serial communication parameters were also verified.

Result

- Stable serial communication
 - Continuous command reception
-

Problem 5: SSH File Transfer Permission Error

While transferring the command file from the Host PC to the Industrial PC, the SCP command returned a Permission Denied error.

Cause

- Missing destination file
- Incorrect file ownership
- Insufficient permissions

Solution

- The destination directory and `command.txt` file were created manually. File ownership and permissions were updated using Linux commands to allow automatic file transfer.

Result

- Successful SSH communication
 - Automatic command synchronization
-

Problem 6: Docker Could Not Read Command File

Initially, the C++ robot controller inside the Docker container could not locate the transferred command file.

Cause

- Incorrect file path
- File not shared with Docker
- Wrong directory mapping

Solution

- The command file was placed in the shared `robot_network_config` directory, which is accessible from both the Ubuntu host system and the Docker container. The file path inside the C++ program was updated accordingly.

Result

- Docker successfully accessed the command file
 - Real-time command reading achieved
-

Problem 7: Robot SDK Compilation Errors

Several compilation errors occurred while building the C++ robot controller using the Syncro5 SDK.

Cause

- Missing libraries
- Incorrect CMake configuration
- Header file dependency issues

Solution

- The CMake configuration file was updated to include the required SDK libraries and header files. Missing dependencies were linked correctly before compilation.

Result

- Successful compilation
 - Executable robot controller generated
-

Problem 8: Robot Communication Failure

The collaborative robot occasionally failed to establish communication during startup.

Cause

- Network configuration issues
- Incorrect controller initialization
- Communication timeout

Solution

- Robot network settings were verified, the communication sequence was restarted, and the Docker container was reinitialized before running the application.

Result

- Stable robot connection
 - Successful initialization
-

Problem 9: Continuous Repeated Commands

Initially, the Python communication program transferred the same command repeatedly, creating unnecessary network traffic.

Cause

- Commands transmitted continuously
- No duplicate command filtering

Solution

- A command comparison mechanism was introduced. The Python program transfers the command file only when a new command differs from the previous one.

Result

- Reduced network traffic
 - Improved communication efficiency
-

Problem 10: Gripper Overheating

During repeated testing, the Feetech electric gripper displayed an overheating warning and temporarily stopped responding.

Cause

- Continuous open and close operations
- High motor temperature
- Safety protection mechanism

Solution

- Adequate delay was introduced between consecutive gripper operations, allowing the motor to cool naturally before executing the next command.

Result

- Stable gripper performance
 - Safe operation
 - Prevention of hardware damage
-

Problem 11: Robot Did Not Stop Immediately

In the initial implementation, the robot sometimes continued moving briefly after the user's hand returned to the neutral position.

Cause

- Continuous command transmission
- No dedicated stop logic

Solution

- An open-palm gesture was assigned as the STOP command. The C++ controller resets all joint jogging directions to zero whenever the stop command is received.

Result

- Immediate robot stopping
 - Improved operator safety
 - Better user control
-

Problem 12: Integration of Multiple Technologies

The most challenging part of the project was integrating multiple technologies into a single working system. The project involved embedded programming, wireless communication, Python scripting, Linux networking, Docker containers, C++ programming, and industrial robot control.

Solution

The project was developed using a modular approach. Each module was tested independently before system integration. This reduced debugging complexity and simplified fault identification.

Result

Successful integration

Reliable system performance

Easier maintenance and future development

Performance Analysis

Performance analysis is an important stage in evaluating the effectiveness of the Smart Glove for Intelligent Device Control. It measures how efficiently the developed system performs under practical operating conditions. The analysis was carried out by observing gesture recognition accuracy, communication reliability, robot response, system stability, and overall operational performance.

The developed system was tested repeatedly using different hand gestures and robot movements. The results demonstrate that the proposed system is capable of providing reliable real-time control of the Syncro5 Collaborative Robot.

Gesture Recognition Performance

The gesture recognition module was evaluated by repeatedly performing all predefined hand gestures. The flex sensors successfully detected finger movements, and the ESP32 transmitter accurately converted them into corresponding control commands.

The implementation of automatic calibration and Exponential Moving Average (EMA) filtering significantly improved the stability of sensor readings and minimized false detections.

- Observations
- Stable gesture detection
- Reduced sensor noise
- Accurate finger recognition
- Consistent command generation
- Result

The gesture recognition system operated reliably throughout the testing process and produced consistent results under normal operating conditions.

Wireless Communication Performance

The ESP-NOW protocol was used for wireless communication between the transmitter and receiver ESP32 modules. During testing, commands were transmitted successfully without requiring a Wi-Fi router. Communication remained stable over repeated operations with very low delay.

Observations

- Fast command transmission
- Reliable wireless communication
- No noticeable packet loss during testing
- Stable connection between ESP32 modules
- Result

ESP-NOW proved to be an efficient communication protocol for real-time gesture-based robot control.

Robot Response Performance

The robot response was evaluated by observing the time taken for the Syncro5 Collaborative Robot to react after

performing a hand gesture.

The communication chain from the glove to the robot controller introduced only a small delay, allowing smooth and responsive robot movement.

Observations

- Quick robot response
- Smooth joint movement
- Stable gripper operation
- Continuous control without noticeable lag
- Result

The robot responded efficiently to gesture commands, making the interaction feel natural and intuitive.

Communication Reliability

The communication system consisted of multiple stages, including ESP-NOW wireless communication, USB serial communication, SSH file transfer, and Docker-based command processing. Each communication stage was tested repeatedly during system operation.

Observations

- Stable serial communication
- Reliable SSH file transfer
- Successful Docker command reading
- Consistent command synchronization
- Result

No major communication failures were observed after proper system configuration.

System Stability

Long-duration testing was performed to evaluate the stability of the complete system.

The glove continuously generated commands, while the robot executed joint movements and gripper operations repeatedly.

Observations

- Continuous operation without software crashes
- Stable communication throughout testing
- Reliable command execution
- Consistent robot performance
- Result

The system maintained stable performance during extended operation.

Resource Utilization

The software architecture was designed to use system resources efficiently.

The ESP32 handled sensor processing locally, while Python managed communication and C++ controlled the robot.

This modular approach distributed computational tasks across different devices.

Advantages

- Low processor load on ESP32
- Efficient communication
- Fast command execution
- Reduced processing overhead

Overall System Performance

The integrated system was evaluated based on its overall functionality and ease of operation. The smart glove successfully converted natural hand gestures into robot commands, enabling intuitive control of the collaborative robot without the use of conventional input devices.

1. Performance Highlights
2. Real-time gesture recognition
3. Reliable wireless communication
4. Accurate robot movement
5. Stable gripper control
6. Smooth system integration
7. Easy operation

Performance Summary Table

Performance Parameter	Observation	Result
Gesture Recognition	Accurate and stable	Excellent
Flex Sensor Performance	Stable after calibration	Excellent
ESP-NOW Communication	Reliable with low latency	Excellent
Serial Communication	Continuous data reception	Excellent
SSH File Transfer	Successful command synchronization	Excellent
Docker Communication	Correct command access	Excellent

Robot Motion	Smooth and accurate	Excellent
Gripper Operation	Reliable opening and closing	Excellent
System Stability	Stable during continuous testing	Excellent
Overall Performance	Real-time and reliable operation	Excellent

Table 28.1: Overall Performance Evaluation

Advantages Observed During Performance Evaluation

The developed system demonstrated several practical advantages during testing:

Natural and intuitive gesture-based control.

Wireless communication without additional networking hardware.

Low-latency command transmission.

Smooth and accurate robot movement.

Reliable gripper operation.

Modular software architecture for easy maintenance.

Expandable design for controlling other intelligent devices.

Improved human-robot interaction.

Applications

The Smart Glove for Intelligent Device Control is a flexible and scalable system that can be used in various fields where intuitive and wireless control is required. Although the prototype was developed for controlling the Syncro5 Collaborative Robot, the same technology can be extended to many other intelligent devices with minimal modifications.

The major applications of the developed system are described below.

Industrial Automation

The smart glove can be used to control collaborative robots in manufacturing industries. Operators can perform robot movements naturally through hand gestures without using traditional control panels or teach pendants.

Applications

Material handling

Pick-and-place operations

Machine tending

Assembly line automation

Quality inspection

Figure 29.1: Smart Glove in Industrial Automation

(Insert industrial robot image.)

Collaborative Robotics

The developed system provides a natural Human-Robot Interaction (HRI) interface for collaborative robots (Cobots). Operators can easily control robot movements using simple hand gestures, making robot operation more

intuitive and user-friendly.

Applications

Human-robot collaboration

Robot teaching

Laboratory automation

Educational robotics

Figure 29.2: Collaborative Robot Control

(Insert Syncro5 robot image.)

RC Vehicle Control

The gesture recognition system can be adapted to control RC cars, robotic vehicles, and mobile robots. Different hand gestures can be mapped to movement commands such as forward, reverse, left, right, and stop.

Applications

RC cars

Mobile robots

Educational robotic platforms

Research projects

Drone Control

The smart glove can also be integrated with drones for gesture-based flight control. Hand gestures can replace conventional remote controllers for controlling drone movement and navigation.

Applications

Drone navigation

Aerial photography

Surveillance

Search and rescue operations

Medical and Rehabilitation Systems

Gesture-controlled devices can assist patients with limited mobility by providing an easy and contactless method for controlling assistive robotic systems.

Applications

Rehabilitation robots

Assistive robotic arms

Patient support systems

Healthcare automation

Smart Home Automation

The smart glove can be integrated with IoT-based smart home systems to control household appliances using hand gestures.

Applications

Smart lighting

Fan control

Door automation

Home security systems

Smart appliances

Educational and Research Laboratories

The project serves as an excellent educational platform for learning embedded systems, robotics, wireless

communication, computer vision, and automation technologies.

Applications

Robotics laboratories

Engineering education

Research projects

Embedded systems training

Hazardous Environment Operations

The smart glove can be used for remotely operating robots in environments that may be unsafe for humans, reducing direct exposure to hazardous conditions.

Applications

Chemical industries

Nuclear facilities

Firefighting robots

Disaster response

Hazardous material handling

Warehouse and Logistics Automation

Gesture-controlled robots can improve warehouse operations by enabling workers to control robotic systems quickly and efficiently during material transportation and inventory management.

Applications

Warehouse automation

Inventory movement

Package handling

Logistics operations

Defense and Security

The developed technology can be extended for remote operation of robotic platforms used in defense and security applications where direct human involvement is risky.

Applications

Bomb disposal robots

Surveillance robots

Military robotic systems

Border security operations

Limitations

Although the Smart Glove for Intelligent Device Control successfully demonstrates real-time gesture-based control of the Syncro5 Collaborative Robot, the current prototype has certain limitations. These limitations provide opportunities for future improvements and further research.

Limited Number of Gestures

The current system recognizes only a predefined set of hand gestures. Users cannot create custom gestures or dynamically change the gesture mapping without modifying the program.

Single Robot Compatibility

The developed software has been implemented specifically for the Syncro5 Collaborative Robot. To control robots from other manufacturers, the communication interface and control APIs would need to be modified.

Dependence on Flex Sensors

The accuracy of the system depends on the proper functioning of the flex sensors. Incorrect sensor placement, loose connections, or sensor wear may reduce gesture recognition accuracy.

Battery Dependency

The smart glove is powered by rechargeable lithium-ion batteries. Continuous operation depends on battery capacity, requiring periodic recharging during extended use.

Limited Degrees of Motion

The current implementation controls selected robot joints and the gripper. More complex robot movements, coordinated trajectories, and simultaneous multi-axis operations have not yet been implemented.

Communication Delay

Although the communication delay is very small, the complete communication chain (ESP-NOW → Receiver ESP32 → Host PC → SSH → Industrial PC → Docker → Robot) introduces a slight latency compared to direct robot control.

Indoor Operating Environment

The wireless communication was tested under laboratory conditions. Performance may vary in environments with strong electromagnetic interference or significant physical obstacles.

No User Authentication

The present system does not include user authentication or access control. Any authorized transmitter programmed with the correct communication settings could potentially send commands to the receiver.

Limited Error Recovery

The current software performs basic error handling. Automatic recovery from communication failures, hardware faults, or unexpected system interruptions is limited and may require manual intervention.

Industrial Validation

The developed system has been successfully tested in a laboratory environment using the Syncro5 Collaborative Robot. However, long-term industrial deployment under continuous production conditions has not yet been evaluated.

Future Scope and Enhancements

The Smart Glove for Intelligent Device Control has demonstrated the feasibility of controlling a collaborative robot through hand gestures. Although the current system successfully controls the Syncro5 Cobot, its modular architecture allows it to be expanded into a more advanced and intelligent human-machine interaction system. The following enhancements can be incorporated in future versions of the project.

Artificial Intelligence Based Gesture Recognition

Future versions can integrate Artificial Intelligence (AI) and Machine Learning (ML) algorithms to automatically recognize a larger variety of hand gestures. This would eliminate the need for manually programmed gesture combinations and improve recognition accuracy.

Control of Multiple Intelligent Devices

The smart glove can be expanded to control multiple devices such as collaborative robots, robotic arms, RC cars, drones, mobile robots, wheelchairs, and smart home appliances using different operating modes.

Wireless Mobile Application

A dedicated Android or iOS application can be developed to monitor the glove, configure gestures, display battery status, and control connected devices remotely without requiring a computer.

Cloud-Based Monitoring

Cloud integration can be added to monitor robot status, gesture history, and system logs remotely. This would enable real-time monitoring and data analysis from anywhere with an internet connection.

Voice and Gesture Hybrid Control

Voice recognition can be combined with gesture control to provide a more natural and flexible Human-Machine Interface. Users could issue voice commands while using hand gestures for precise movement.

Haptic Feedback System

Vibration motors or haptic actuators can be integrated into the glove to provide tactile feedback whenever a command is successfully executed or when the robot encounters an obstacle or error.

Computer Vision Integration

A camera-based vision system can be integrated with the smart glove to enable object detection, object tracking, and autonomous pick-and-place operations. This would allow the robot to identify and manipulate objects more intelligently.

Internet of Things (IoT) Integration

The system can be connected to IoT platforms for remote device monitoring, automation, and smart environment control. Users could operate intelligent devices over the internet from any location.

Enhanced Safety Features

Future versions can include advanced safety mechanisms such as obstacle detection, collision avoidance, emergency shutdown, user authentication, and encrypted communication to improve operational safety.

Wearable Design Improvements

The smart glove can be redesigned using flexible electronic components, lightweight batteries, and custom PCBs to improve comfort, durability, and long-term usability.

Industrial Deployment

The developed prototype can be optimized for industrial applications such as assembly lines, warehouse automation, material handling, quality inspection, and collaborative manufacturing environments.

Medical and Rehabilitation Applications

The glove can be adapted for rehabilitation exercises, assistive robotic devices, prosthetic hand control, and healthcare applications requiring natural gesture-based interaction.

References and Appendix

References

The following books, software documentation, research resources, and official websites were referred to during the design, development, implementation, and testing of the **Smart Glove for Intelligent Device Control** project.

Books

Craig, J. J., *Introduction to Robotics: Mechanics and Control*, Pearson Education.
Siciliano, B., Sciavicco, L., Villani, L., & Oriolo, G., *Robotics: Modelling, Planning and Control*, Springer.
Alciatore, D. G., & Hstand, M. B., *Introduction to Mechatronics and Measurement Systems*, McGraw-Hill.
Bolton, W., *Mechatronics: Electronic Control Systems in Mechanical and Electrical Engineering*, Pearson.

Official Documentation

Python Software Foundation. *Python Documentation*.
Arduino. *Arduino IDE Documentation*.
Espressif Systems. *ESP32 Programming Guide*.
Espressif Systems. *ESP-NOW Documentation*.
OpenCV Documentation.
MediaPipe Documentation.
Docker Documentation.
Ubuntu Documentation.
CMake Documentation.
GNU GCC/G++ Documentation.
Addverb Technologies. *Syncro5 Robot SDK Documentation*.
Feetech Servo and Gripper Documentation.

Online Learning Resources

GeeksforGeeks
Stack Overflow
GitHub
Arduino Community
Espressif Developer Resources

Software Used

Ubuntu 22.04 LTS
Docker
Python 3.10
Arduino IDE

Visual Studio Code
CMake
GCC / G++
SSH
Syncro5 Robot SDK

AI Assistance

During software development, debugging, documentation, and technical understanding, AI-based assistance was used as a learning and productivity tool for code explanation, troubleshooting, report drafting, and improving project documentation.

APPENDIX

The appendix contains supporting information that complements the main report. These materials provide additional technical details, diagrams, source code, and experimental results used during the development of the project.

Circuit Diagram

Include the complete circuit diagram of the Smart Glove showing:

ESP32 Transmitter

Five Flex Sensors

10 k Ω Resistors

Battery Pack

Charging Module

Power Connections

Figure A.1: Smart Glove Circuit Diagram

(Insert circuit diagram.)

Connection Diagram

Include wiring diagrams for:

Flex Sensor to ESP32 connections

ESP32 Transmitter wiring

ESP32 Receiver wiring

Battery and charging module connections

USB connection to Host PC

Figure B.1: Complete Connection Diagram
(Insert wiring diagram.)

Source Code

Include the complete source code used in the project.

C.1 ESP32 Programs

ESP32 MAC Address Program

ESP32 Transmitter Program

ESP32 Receiver Program

C.2 Python Programs

Serial Communication Program

SSH Communication Program

Gesture Control Program

C.3 C++ Programs

Robot Controller

Command Reader

Glove Robot Control

Teaching Mode (if included)

Appendix D: Project Snapshots

Include photographs and screenshots taken during different stages of the project.

Suggested sequence:

Completed Smart Glove

Internal view of glove electronics

ESP32 Transmitter

ESP32 Receiver

Flex sensor installation

Battery and charging module

Host PC terminal

Docker terminal

SSH communication

Python program execution

Arduino Serial Monitor

Gesture detection interface

Robot initialization

Robot movement

Gripper opening
Gripper closing
Complete experimental setup
Final working system

Snapshots

Include a summary table of the hardware and software used.

Hardware

Syncro5 Collaborative Robot
Feetech Gripper
ESP32 Transmitter
ESP32 Receiver
Five 2.2-inch Flex Sensors
Leather Glove
Lithium-ion Batteries
Charging Module
USB Cable
Industrial PC (Odyssey)
Host PC
Jumper Wires
10 k Ω Resistors

Software

Ubuntu 22.04 LTS
Docker
Python 3.10
Arduino IDE
C++
CMake
GCC / G++
SSH
Syncro5 Robot SDK

Conclusion

The **Smart Glove for Intelligent Device Control** was successfully designed, developed, and implemented as a real-time gesture-based human-machine interaction system. The project demonstrates that natural hand gestures can be effectively used to control intelligent devices without relying on conventional input devices such as keyboards, joysticks, or teach pendants.

The developed system integrates **flex sensors, ESP32**

microcontrollers, ESP-NOW wireless communication, Python, SSH, Docker, C++, and the Syncro5 Robot SDK into a unified control architecture. The smart glove accurately detects finger movements, converts them into predefined commands, and transmits them wirelessly to control the Syncro5 Collaborative Robot. The robot successfully performs joint movements and gripper operations according to the user's hand gestures.

Throughout the project, every module was individually tested and then integrated into the complete system. Stable communication was achieved between the glove, the Host PC, the Industrial PC, the Docker environment, and the collaborative robot. The system demonstrated reliable gesture recognition, low communication delay, smooth robot motion, and consistent overall performance during experimental testing.

During development, several technical challenges such as sensor noise, gesture instability, wireless communication issues, Docker integration, SSH communication, and robot controller configuration were successfully identified and resolved. These experiences strengthened the overall design and improved the reliability of the final system. The project also highlights the potential of wearable technology in modern robotics and automation. Its modular architecture allows future expansion for controlling various intelligent devices, including robotic arms, RC vehicles, drones, smart home appliances, assistive robots, and IoT-based systems. With the integration of Artificial Intelligence, computer vision, and cloud technologies, the proposed system can evolve into a more intelligent, adaptive, and versatile human-machine interface.

In conclusion, the **Smart Glove for Intelligent Device Control** successfully achieves the objectives of the project by providing an

intuitive, wireless, and efficient method for controlling collaborative robots. The developed prototype demonstrates the practical feasibility of gesture-based device control and serves as a strong foundation for future research and industrial applications in robotics, automation, and intelligent control systems.